AUTHBC: Feasibility Boundaries for Authenticated UAV Telemetry — An Exclusion Bound and a Capacity Envelope

Mohamed A. Farouk[1], Mahmoud Abdelaziz[1*], Ahmed F. Darwesh[1], Ashraf D. Elbayoumy[1]

[1*]Department of Communication Engineering, Military Technical College, Cairo, Egypt.

*Corresponding author(s). E-mail(s): mkaremm@mtc.edu.eg;
Contributing authors: farouk.m.ashraf@gmail.com;
ahmed.darwesh@alumni.ucalgary.ca; adiaa@mtc.edu.eg;

Abstract

Unmanned Aerial Vehicles (UAVs) that broadcast signed, hash-chained telemetry spend more bytes on authentication than on data: a 64 B signature and a 32 B chain link accompany a 24.0 B record. AUTHBC is a frame format, sender and receiver in which one signature and one chain link cover b records, yet every frame verifies alone. Batch size, encoding and signature scheme are selected jointly under five constraints: freshness, verifiability, channel capacity, receiver CPU and payload size. Measured frames, ns-3 simulation and metered sender energy give two feasibility boundaries. Where the frame fits (50 records/s, 100 ms deadline), AUTHBC sends 43.25 B per record against 145.77 B for per-record signing, and one simulated 802.11a collision domain carries 124 nodes instead of 35 at 5% frame loss: the cost that binds is the frame, not the authenticator. Batching gains nothing unless record rate times deadline exceeds two. Where it does not fit, eight of the twelve EU863-870 LoRaWAN data rates cannot carry it. Capacities are simulated; when the swarm spreads, fewer neighbors are in reach, but the gain stays 3.46–3.63-fold, moving or not. On hardware, contention was measured between two radios only.

Keywords: FANET, UAV telemetry, broadcast authentication, digital signatures, hash chain, batching, frame aggregation, age of information, formal verification, IEEE 802.11 DCF, LoRaWAN, ns-3

1 Introduction

Flying Ad-hoc Networks (FANETs) let swarms of Unmanned Aerial Vehicles (UAVs) coordinate without central infrastructure. Each vehicle reports its position, velocity and status to its neighbors many times per second. When these reports must later serve as evidence — of where each vehicle was, and of what it claimed — every report must be attributable to its sender, and the sequence must be tamper-evident. The standard construction signs every record and links it to its predecessor by a hash. Once the payload is compact, however, the protection becomes larger than the data it protects. Our earlier comparison of encoding formats showed that canonical Concise Binary Object Representation (CBOR) reduces an 18-field telemetry message from 358 B in JavaScript Object Notation (JSON) to 130 B [1]. The compactly encoded position, velocity and status record of this work occupies about 24.0 B, whereas an Ed25519 or Elliptic Curve Digital Signature Algorithm (ECDSA) signature occupies 64 B and a SHA-256 chain link 32 B. On a shared radio channel these bytes become airtime, and airtime determines how many neighbors can be heard.

Current practice authenticates each message individually. MAVLink 2 appends a 13 B symmetric tag to each message [2], and vehicular standards attach an ECDSA signature, and periodically a certificate, to each one [3]. Existing stream-signing schemes [4–6] amortize the cost of signing across packets without reducing the number of frames that a sender places on the channel. Conversely, aggregate-signature schemes [7] reduce the work of the verifier without reducing what each signer transmits. Letting several records share one frame, one signature and one chain link is an established alternative, and its price is known in kind: records wait for the frame to fill, and a lost frame loses all of them [8]. What prior work does not provide is the accounting: what a frame must carry so that a receiver can verify it without any other frame, on which links such a frame does not fit at any batch size, which constraint caps the batch where it fits, and what the saved bytes buy on a contended channel in nodes and not in percent. This gap leaves system designers without data-driven guidance for selecting batch size, encoding and signature scheme jointly.

This paper presents AUTHBC (AUTHenticated telemetry for BlockChain-style ledgers) as a frame format, a sender and a receiver. The system was implemented, measured and searched over its parameters. Its central observation is that on a contended broadcast channel the cost that binds is the frame, not the authenticator. The main contributions are:

- **Byte and capacity gain.** One signature and one chain link per four records cut the bytes per record by 70.3% and raise the simulated capacity of one 802.11a collision domain from 35 to 124 nodes. A step-by-step comparison shows that most of the gain comes from sending fewer frames (Sections 3.1 and 3.4).
- **Capacity law and mechanism.** An airtime capacity law with one fitted constant reproduces the simulated capacities. A contention model with nothing fitted explains the constant and predicted cases that had not been simulated, other batch sizes and loss levels among them (Section 3.5).
- **Binding constraints.** Freshness, not the Maximum Transmission Unit (MTU), caps the batch at four records. A frame that depends on its predecessor misses the

verifiability target, so every frame carries a key record. A Rust receiver serves 290 nodes per core, so the channel, not the processor, binds (Sections 3.2–3.6).

- **Exclusion bound.** Eight of the twelve EU863-870 LoRaWAN data rates cannot carry one signed, chained frame: five because the signature alone does not fit, and three because of the header of this format. The five conditions of the bound, and what each relaxation buys, are stated (Section 3.11).
- **Security model and defenses.** Origin, integrity, replay protection, equivocation evidence, gap detection and continuity are proven in the Tamarin prover, in three separate models. Two defenses are measured: a timestamp acceptance window against delayed frames, and an 8 B group message authentication code (MAC) against floods from outside the swarm (Section 3.10).

Batching several records under one signature is not new. What this paper adds is the accounting — which links can carry a self-verifying frame, which constraint caps the batch, and what the saved bytes buy in nodes on a contended channel — together with a measured implementation. AUTHBC introduces no new cryptographic primitive. Each result states whether it is measured, simulated or only modeled; Section 4.4 collects the limitations, and Table 16 summarizes which configuration to use.

1.1 Related work

Our work intersects per-message authentication, stream signing, aggregate and batch verification, hash-linked records, compact encoding for small payloads, and broadcast channel models.

Per-Message Authentication: MAVLink 2 signing appends a link identifier, a 48-bit timestamp and a 48-bit truncated SHA-256 tag computed with a shared secret [2]. This mechanism authenticates a *link*: any holder of the key can produce any message, so a record cannot later be attributed to one vehicle. Rotta and Mykytyn likewise attach a keyed hash (HMAC) and a timestamp to each broadcast telemetry message of a swarm [9]. Vehicular broadcast uses public-key signatures: ECDSA over each message, with the signer's certificate in some messages and an eight-octet digest of it in the others [3, 10]. We adopt this trust model, and its per-message cost is the baseline of our evaluation.

Stream Signing: To amortize one signature over many packets, Gennaro and Rohatgi sign the first packet and chain the others by hashes [4]; one lost packet breaks the chain. Wong and Lam sign a hash tree over a block and send in every packet the hashes needed to reach the root [5]. Each packet then verifies alone, and the signing cost is amortized but the bytes are not. EMSS tolerates loss by carrying several hashes per packet and a periodic signature packet. TESLA replaces the signature by a MAC whose key is disclosed later [6, 11], and TBRD applies TESLA to UAS Remote ID with about half the overhead of signatures [12]. Erasure codes spread the hashes and the signature of a block over its packets, so that the block verifies when enough of them arrive, for 10 B per packet in blocks of sixteen at 5% loss and a latency of one block [13]. All these schemes spread *one authenticator across packets*, so packets depend on each other or on a later one. AUTHBC places *several records in one packet*, which a small telemetry record allows. Each frame then stays self-verifying,

and the cost moves to latency at the sender. TESLA instead delays verification at the receiver, and a disclosed MAC key gives no non-repudiation, which a ledger requires. Section 3.12 places these schemes at our operating point.

Aggregate and Batch Verification: BLS signatures aggregate [14, 15], and certificateless aggregate signature (CLAS) schemes for vehicular networks build on such constructions [7]. Zhang *et al.* verify many signatures at once at a roadside unit [16]. Rajasekaran *et al.* batch-authenticate a group of drones with a pairing-based scheme and report a communication cost that grows linearly with the group [17]. All of these reduce the *work* of the verifier. Each message, however, still carries its own signature material as its signer sends it (Section 3.12).

Hash-Linked Records: Linking each record to the hash of its predecessor to make a sequence tamper-evident is due to Haber and Stornetta [18]; blockchain applications on UAVs are surveyed in [19]. We take the chain as given and quantify what it costs on air.

Compact Encoding and Small Payloads: That a signature leaves little room in a small frame is established. Gündoğan *et al.* find 9 B of application data left beside an Ed25519 signature in an IEEE 802.15.4 frame [20]. Post-quantum signatures do not fit one 5G broadcast block [21]. For Remote ID, a 64-octet signature is paged over 25-octet Bluetooth frames with a parity page [22]. This is a form of spreading and recovery that our scope excludes. Automatic Dependent Surveillance–Broadcast (ADS-B) is the extreme case: its 112-bit messages leave 56 bits for data. Proposals for public-key authentication there distribute one signature over several messages that receivers buffer [23]. Compact encodings [24] and delta coding of time series [25] are established, and our earlier evaluation found canonical CBOR byte-identical across 10,000 trials [1]. The AUTHBC frame is a canonical CBOR map.

Channel Models: Broadcast frames are not acknowledged, and their contention window never doubles. The saturation model of Bianchi for the IEEE 802.11 Distributed Coordination Function (DCF) [26] therefore does not apply as it stands; we use the broadcast model of Ma and Chen [27, 28] and validate it against ns-3 [29]. For LoRa, Haxhibeqiri *et al.* model single-gateway scalability [30], and Zirak *et al.* report air-to-air delivery against range on hardware [31]. These models describe the channel alone. Our evaluation applies them to each authentication configuration at its own frame size and frame rate.

2 Methods

2.1 System and threat model

Table 1 lists the notation, and Fig. 1 gives an overview. Each UAV codes, chains and signs its records into frames and broadcasts them on a shared channel. Each receiver decodes, verifies and stores the frames of its neighbors. Five constraints fix the design parameters (Section 2.3).

Network and Traffic: N UAVs share one channel and all hear one another: one collision domain, which we call a neighborhood of N nodes. Each UAV produces Λ telemetry records per second and broadcasts them, without acknowledgments or retransmissions. The primary operating point is $\Lambda = 50$ records/s with a freshness

Table 1 Notation used in this paper.

Symbol	Meaning
N	nodes in one collision domain (a neighborhood)
Λ	records per second per node
$D_{\max}$	freshness deadline
b	records per frame (batch size)
r	key-record interval: one frame in r is self-verifying
p	frame loss probability
V, ϵ	records verified; target $V \geq 1 - \epsilon$
H_f, L, g_a	header, chain link, signature (bytes)
s_k, s_δ	key record, delta record (bytes)
T_{ch}, t_v	channel delay; verification time
$N_{\max}$	capacity: largest N with mean delivery $\geq 1 - \epsilon$
N_{sat}	saturation bound of the broadcast model
f, T	frames per second per node; medium time of a frame
Δ	timestamp acceptance window
M	maximum payload of a link

bound $D_{\max} = 100\,\text{ms}$. This rate is the companion-computer stream rate of PX4 [32]. The pair lies within the 3GPP requirements for direct UAV-to-UAV broadcast: at least 10 messages per second and an end-to-end latency of at most 100 ms (TS 22.125, R-5.2.2-010 and R-5.2.2-011 [33]). A relaxed point of 20 records/s and 250 ms is reported for comparison.

Telemetry Record: Each record carries nine fields, all fixed-point integers:

- `ts` – timestamp in milliseconds
- `lat`, `lon` – latitude and longitude in degrees$\times 10^7$
- `alt` – altitude in centimeters
- `vel_x`, `vel_y`, `vel_z` – velocity components in cm/s
- `battery` – battery level in percent
- `mode` – flight mode (enumeration)

In the ledger, each record also carries the identifier of its sender, its sequence number and the hash of its predecessor.

Ledger: Each UAV keeps an append-only chain of its own records, $prev_hash_i = \text{SHA-256}(rec_{i-1})$, and stores the verified records it hears from others. No consensus layer is included; the subject of this paper is authenticated dissemination. A ledger can be audited from the uploaded frames alone, with the public keys of the senders (Section 3.10).

Adversary: The adversary controls the radio: it reads, injects, replays, modifies, delays and drops frames, and may operate many radios. It holds no private key of an honest UAV.

What Is Protected: (i) *Origin and integrity:* a stored record was produced by the UAV it names and was not altered; a public-key signature is checked before anything is stored. (ii) *Replay:* sequence numbers strictly increase per sender, so a repeated frame is refused. (iii) *Equivocation:* two validly signed records for one sequence number are detected, and the two frames that carried them are kept. The signature covers

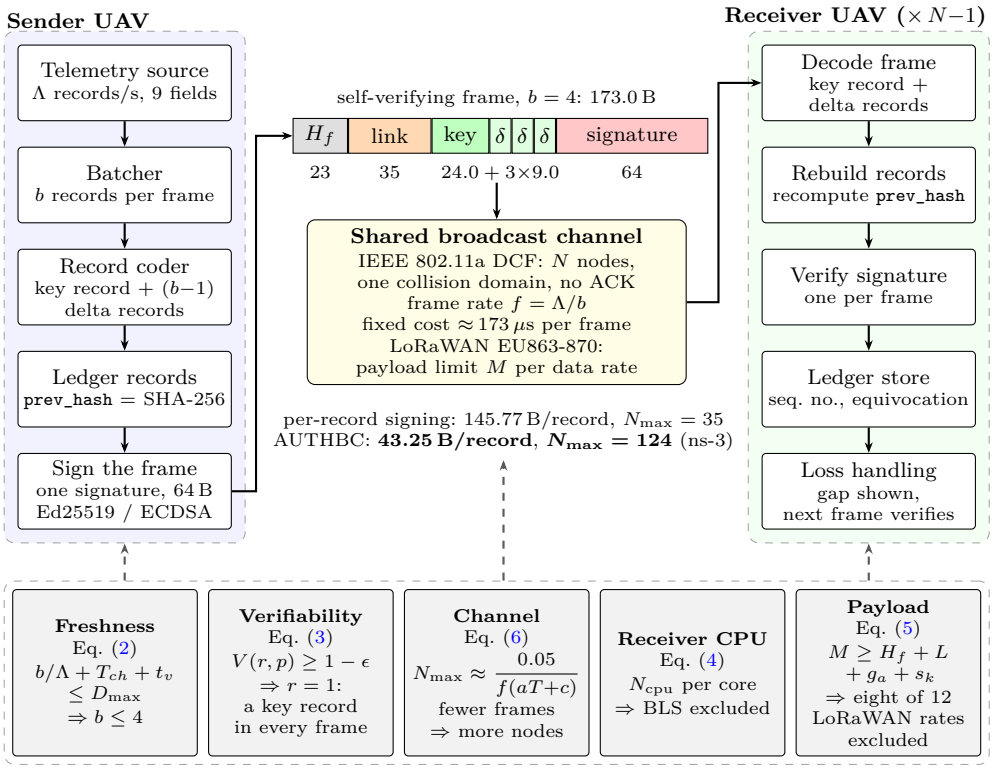


Fig. 1 System overview: the sender pipeline, the four-record frame with its field sizes in bytes, the shared channel, the receiver pipeline and the five design constraints.

a frame, so a record is evidence only with its frame; a third party checks the pair with the sender's public key alone. Since only a holder of the sender's key can sign either record, an equivocation identifies a misbehaving or compromised UAV. (iv) *Continuity:* omission or reordering within what a receiver holds is detected through the chain. (v) *Bounded-delay freshness:* while GNSS time is available, a receiver refuses a frame whose timestamp is more than Δ older than its own clock (Section 3.10).

What Is Not Protected: Telemetry is sent in clear. Jamming is not resisted: loss appears as the robustness measure V below, not as a security property. Flooding is resisted only from outside the swarm. An 8 B group MAC, under a key that all members hold, refuses an outsider's frame before it is decoded. A member holding that key can still make a receiver verify forged frames (Section 3.10). A frame delayed by less than Δ is accepted, authentic and in order, and so is a later one once GNSS time has been lost for longer than the window allows. A UAV with a valid key can sign false data; only inconsistent falsehood is detectable. Key distribution, revocation and admission are assumed: receivers hold the public keys of the senders, and the bytes of delivering certificates are charged separately (Section 3.7). Senders are not anonymous.

Table 2 Byte budget of a frame, field by field, in the record-linked and the frame-linked format.

Field	Content	Record-linked	Frame-linked
(map)	CBOR map header	1	1
`v`, `t`	version; frame type	6	4
`src`	sender identifier, 16 bit	7 (5)	4 (2)
`base_seq`	sequence no. of first record	14 (10)	6 (2)
`n`	records in the frame	3	2
`recs`	key, length of records	6	3
`auth`	key, length of signature	7	3
header H_f †		**44** (38)	**23** (17)
chain link		32 per record	35 per frame
signature g_a (Ed25519, ECDSA)		64	64
key record s_k		59.9*	24.0
delta record s_δ, 50 ms apart		44.0*	9.0

† five-digit sender identifier, one hour into a 50 Hz flight; in parentheses: sender 0, first frame of a flight.
* includes the 32 B chain link of that record.

Formal Analysis: Properties (i)–(iv) and the detection of gaps are proven in the Tamarin prover [34] for unboundedly many frames, and (v) is shown to hold with the window and to fail without it. The proofs use three separate models, because one model with both the chain and the clock did not terminate. The three models are not composed. That the properties hold together for the full protocol is argued, not proven: the receiver of the model without chain and clock checks less than the full receiver, so it accepts a superset of what the full receiver accepts. We also assume canonical encoding to be injective: two different sets of records never encode to the same bytes.

Verifiability Metric: V is the probability that a record a UAV generates is received, decoded and verified by a given neighbor. We require $V \geq 1 - \epsilon$ and take $\epsilon = 0.05$ throughout.

2.2 The AUTHBC frame

Frame Structure: A frame is one canonical CBOR map [24] with integer keys (Table 2, Fig. 1); Appendix A gives the complete layout. The fields `src` and `base_seq` name the sender and the sequence number of the first record. The field `link` is the hash of the record before it, `recs` is the record stream, and `auth` is one signature. A record in the stream consists of nine variable-length integers. The first record of a frame is a *key record*, carrying absolute values (24.0 B on average). Each later one is a *delta record*, the difference from the record before it (9.0 B, the floor of the format, at the 20 and 50 ms spacings of our two operating points). The sender identifier and sequence number are not repeated in the records. A frame that can be decoded and verified without any other frame is *self-verifying*; every AUTHBC frame is.

Signed Content and Verification: The signature covers not the record stream but the canonical ledger records, each with its own `prev_hash`. The receiver first rebuilds them: the first `prev_hash` is `link`, and each later one is the hash of the record just rebuilt. Only then does it check the signature. A frame that cannot be decoded therefore cannot be verified, and nothing unverified is stored. Each verified record is offered to the ledger store. The store refuses a sequence number that is not newer, keeps an equivocating pair as evidence, and flags a link that contradicts a record it holds. The decoding state advances only after a frame has been verified and stored, so a replayed frame cannot desynchronize a stream. Flipping any single bit of a frame makes verification fail; all 1 392 positions of a four-record frame were tried.

Behavior Under Frame Loss: When a frame is lost, the next one still decodes and verifies, because it starts with a key record. Its `link` then names a record that this receiver does not hold. The gap shows in the sequence numbers, and the link is stored unchecked, to be checked by any party that holds the missing record. A late copy of the lost frame is refused, since sequence numbers must increase; the gap is not repaired.

One Chain Link per Frame: The ledger still chains every record, but only the first link of a frame travels; the receiver recomputes the others. Inside a frame, the order and content of records therefore rest on the frame signature alone. In the record-linked format below, every record also carries its own hash on air. A frame is signed and accepted as a unit, so those hashes are redundant and a receiver loses no check. However, this leaves one protection mechanism inside a frame instead of two.

The Record-Linked Format: For comparison we keep the format we built first, the *record-linked format*: text keys, and in every record its own chain link, sender identifier and sequence number. The format described above, with integer keys and one chain link per frame, is the *frame-linked format*; it is 42% smaller. The record-linked header is 44 B, of which 29 B are key names. That format is implemented but no longer developed. Its sizes in this paper are estimated from separately measured components, and its frames with delta records were never produced; such sizes are marked wherever they appear.

Design Parameters: A configuration is defined by four parameters:

- *Batch size* b – the number of records in a frame.
- *Signature placement* – a signature covers each record (*per-record signing*; `auth` is then an array of b signatures) or the whole frame (*per-frame signing*).
- *Signature scheme* – Ed25519 [35] and ECDSA P-256 [36] at 64 B, or BLS12-381 [14] at 96 B in the library used.
- *Key-record interval* r – one frame in r starts with a key record, and the others start with a delta record against the previous frame.

Throughout, *AUTHBC* denotes the frame-linked format with $b = 4$, one signature per frame, delta records and $r = 1$. *Per-record signing* with one record per frame ($b = 1$) is the baseline.

2.3 Constraints

Five constraints decide which configurations are feasible: bytes, freshness, verifiability, channel capacity and receiver CPU select the configuration, and payload decides where a frame can be sent at all.

Bytes: With header H_f, link field L, signature g_a, key record s_k and delta record s_δ, a self-verifying frame of b records with one signature costs, per record,

$$B(b) = \frac{H_f + L + g_a + s_k + (b-1)\, s_\delta}{b}. \tag{1}$$

$B(1)$ is per-record signing: one record, one signature, one frame. Signing each record of a shared frame replaces g_a by $b\, g_a$. Equation (1) is exact to within a byte of CBOR length prefix; for the frame-linked format we report the lengths of frames produced by the implementation.

Freshness: The oldest record in a frame waits for the frame to fill. Adding the channel-access and airtime delay T_{ch} and the verification time t_v,

$$D(b) = \frac{b}{\Lambda} + T_{ch} + t_v \;\le\; D_{\max}. \tag{2}$$

Fill time dominates. At both operating points $\Lambda D_{\max} = 5$: five records fill exactly $D_{\max}$ and leave nothing for the channel, so $b \le 4$ whatever the encoding or the scheme.[1] The 1500 B MTU would admit 31 records in the record-linked format and 151 in the frame-linked one. Because the whole sampling interval is counted, even one record misses the deadline when $\Lambda D_{\max} \le 1$.

Verifiability: If one frame in r is self-verifying, frame j of a group verifies only if it and the $j-1$ frames before it arrive. With independent frame loss p,

$$V(r,p) = \frac{1}{r}\sum_{j=1}^{r}(1-p)^j, \tag{3}$$

which equals $1-p$ at $r=1$ and is smaller for every $r>1$. A target $V \ge 1-\epsilon$ with $\epsilon \le p$ therefore admits $r=1$ only. The same argument forbids splitting a signed unit over n frames, which then verifies with probability $(1-p)^n$.

Channel: For a configuration — a frame size and a frame rate per node — the capacity $N_{\max}$ is the largest neighborhood in which the mean delivered fraction of frames is at least $1-\epsilon$. It is obtained by simulation (Section 2.7). As a conservative companion we report N_{sat}, the largest N whose offered frame rate does not exceed the saturation throughput of the broadcast DCF model [28].

Receiver CPU: A receiver verifies the frames of every neighbor. With k signatures per frame, verification time t_v and hash time t_h, one neighbor costs $(\Lambda/b)\, k\, t_v + \Lambda\, t_h$

[1] Equation (2) counts the whole filling period b/Λ, since a record may describe a state up to one sampling interval old. With b/Λ, the bound is on the *peak age* of information [37] at a receiver that loses no frame: the delay of the newest record of a frame plus the interval to the next frame. Counting only the wait $(b-1)/\Lambda$ would admit $b=5$ and 36.4 B per record.

Table 3 The two experimental platforms.

Parameter	Simulation Host	Embedded Board
Platform	Intel Core i5-14400F	Raspberry Pi 4 Model B Rev 1.4
CPU	x86-64	Cortex-A72, 4 cores, 1.8 GHz
OS	Linux (WSL2)	Debian 12 (bookworm)
libc	glibc 2.39	glibc 2.36
Python	3.12.3	3.12.13
Role	simulation, analysis	timing, energy, radio

seconds of CPU per second, and one core serves

$$N_{\mathrm{cpu}} = 1 + \left\lfloor \frac{1}{(\Lambda/b)\, k\, t_v + \Lambda\, t_h} \right\rfloor \tag{4}$$

nodes, itself included.

Payload: A self-verifying frame must fit the maximum payload M of the link:

$$M \;\geq\; H_f + L + g_a + s_k. \tag{5}$$

2.4 Implementation and platforms

Formats, sender, receiver and ledger store form one Python code base with four record encodings (JSON, CBOR, MessagePack and delta) and three signature schemes behind common interfaces. All components use standard libraries in their default configurations, without hand-tuned optimization:

- cbor2 v5.8.0: canonical encoding [24] of frames and of the ledger records that a signature covers.
- cryptography v45.0.7: Ed25519 and ECDSA P-256 signatures; SHA-256 is that of the Python standard library.
- blspy v2.0.3: BLS12-381 signatures.
- ns-3.48 [29]: channel simulation for IEEE 802.11a and 802.11p, and, with its LoRaWAN module, for the low-rate links.
- Rust with ed25519-dalek: a second receiver and sender, byte-identical to the Python codec on 10 000 frames.

Each signature scheme is checked against published test vectors (for Ed25519, those of [38]). Table 3 lists the two platforms; the board runs with the `performance` frequency governor.

2.5 Telemetry workload

Records come from a seeded random-walk generator: 30 seeds of 1000 records at 50 ms spacing, that of the relaxed point. The delta record sits at the floor of the format, 9.0 B, at both spacings (Section 3.9). Header fields are sized for a five-digit sender

identifier one hour into a 50 Hz flight. Section 3.9 checks the generator against flight logs.

2.6 Timing and energy measurement

Timing: Signing, verification, hashing and encoding are timed on the Raspberry Pi 4 and reported as medians: Ed25519 sign/verify 88/259 μs, ECDSA 136/327 μs, BLS 2.73/8.61 ms, and SHA-256 2.75 μs.

Energy: Sender energy is metered with an INA219 sensor on the supply rail, with five repetitions per configuration and idle power subtracted. The record-linked format was metered on one board; the frame-linked format, in Python and in Rust, on a second board and sensor of the same types.

2.7 Channel simulation

The channel is simulated in ns-3.48 [29]: IEEE 802.11a at 6 Mb/s, ad hoc mode and broadcast frames, with all nodes in range at equal received power, so there is no capture and no hidden terminal. Each node sends one frame every b/Λ seconds, at an instant drawn uniformly within each period, so that the relative phases of the nodes change from frame to frame. A strictly periodic source — one that sends each frame as soon as it fills, as Eq. (2) assumes — freezes every phase for a whole run: a run then samples one phase configuration, in which a few nodes may collide in every period while the others lose nothing. The two sources deliver the same on average: over 14 points and 1 260 runs, the means differ by 0.0004 ± 0.0009. The strictly periodic one varies 6–23 times more from run to run (Section 4.4). Capacity depends on the frame rate and length, not on where in its period a frame is sent. Capacities are therefore simulated with the randomized source and freshness is evaluated with send-at-fill timing; the ages of the randomized source run half a period longer (Section 3.9). Table 4 gives $N_{\max}$ as the largest simulated neighborhood whose mean delivery over 30 seeds is at least 0.95, with its bootstrap interval. The figures and Tables 10 and 15 give the crossing of 0.95, interpolated between the two simulated sizes around it, from the same runs: 124.5 [124.0, 125.0] for AUTHBC and 35.3 [35.1, 35.4] for per-record signing.

2.8 Statistical analysis

A run lasts 20 s, and each point comprises 30 runs. $N_{\max}$ is the largest simulated N whose mean delivered fraction is at least 0.95, with every smaller simulated N also passing. Its interval is the 2.5–97.5 percentile of that quantity over 10 000 bootstrap resamples of the seeds. An interval of a mean over seeds is a Student-t interval, and an interval of an interpolated capacity is an expanded percentile bootstrap interval [39]. With a known truth and 30 seeds, the plain percentile interval covered 92–94% of the time, and these intervals 94–95%. The saturation models were validated separately against saturated ns-3 runs (Section 3.5). Where an experiment tests a prediction, the prediction was fixed before the experiment ran, and misses are reported as misses.

The main evaluation uses one collision domain of static nodes. Section 3.9 repeats it for a swarm spread over a wider area, with hidden stations, and for a moving swarm.

Table 4 Step-by-step comparison at 50 records/s and $D_{\max} = 100$ ms: frame size, bytes per record with and without certificates, saturation bound N_{sat}, simulated capacity $N_{\max}$ with its 95% interval, and receiver CPU share.

	Configuration	b	Frame (B)	B/ record	With certificates: one per 5 frames	With certificates: one per 500 ms	N_{sat}	$N_{\max}$	CPU[2]
Record-linked format: text keys, a chain link in every record[1]									
1	per-record signing	1	174.3	174.25	213.05	188.41	18	32 [32, 32]	41%
2	sign every record, share the frame	4	565.0	141.25	150.95	149.41	24	56 [56, 56]	72%
3	one signature per frame	4	373.0	93.25	102.95	101.41	30	76 [75, 76]	25%
4	… and delta records	4	299.9	74.96	84.66	83.12	34	88 [88, 89]	29%
Frame-linked format: integer keys, one chain link per frame									
5	per-record signing	1	145.8	145.77	184.57	159.93	20	35 [35, 35]	45%
6	sign every record, share the frame (delta records)	4	372.0	93.00	102.70	101.16	30	76 [76, 77]	98%
7	one signature per frame, no delta coding[1]	4	218.0	54.50	64.20	62.66	42	108 [107, 109]	36%
8	one signature per frame, delta records **(AUTHBC)**	4	**173.0**	**43.25**	52.95	51.41	52	**124** [124, 125]	42%

[1]Size estimated from measured components; rows 5, 6 and 8 are measured frames.

[2]Share of one receiver core spent verifying and hashing at $N_{\max}$.

Capacities are obtained by simulation, and contention was measured on hardware between two radios only (Section 3.5).

3 Results

Terminology: Bytes per record is the on-air length of a frame divided by the number of records it carries, $B(b)$ of Eq. (1). The capacity $N_{\max}$ is the largest neighborhood with a mean delivered fraction of at least 0.95 in ns-3, and N_{sat} is the saturation bound of the broadcast model. The capacity ratio is the $N_{\max}$ of AUTHBC divided by that of per-record signing. It depends on the format and is therefore always quoted with the format. Sections 3.1–3.8 give the results on 802.11, then come robustness, security, the low-rate links and the comparison with published schemes.

3.1 Bytes, step by step

Table 4 separates the steps that per-record signing combines. It shows three things:

1. *Sharing the Frame:* When four records share a frame but each keeps its signature (rows 2 and 6), the frame-linked format falls from 145.77 B per record to 93.00 B. The header and the chain link are paid once per four records, and the three records after the first are delta records.

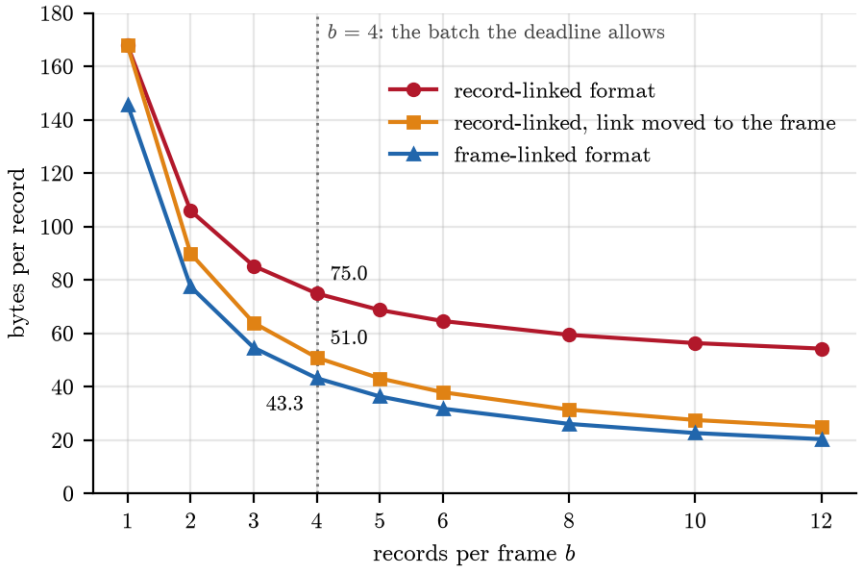


Fig. 2 Bytes per record against batch size, by chain-link position (delta records throughout); record-linked values are estimated from measured components, frame-linked values are measured. With a link in every record, even 12 records per frame cost more than the frame-linked format at $b = 4$.

2. *Sharing the Signature:* One signature per frame yields **43.25 B, 70.3% below per-record signing**. Row 7 is the same frame without delta coding, at 54.50 B.
3. *Position of the Chain Link:* In the record-linked format, sharing the frame gives 141.25 B (from 174.25 B), sharing the signature 93.25 B, and delta coding 74.96 B, 57.0% below its per-record signing. It ends higher because each of its records carries its own 32 B link, which no batch size amortizes.

Fig. 2 isolates the position of the link. Moving the link of the record-linked format from the record to the frame saves $32\,(1-1/b)$ B per record: 24 B at $b = 4$, which gives 51.0 B. The frame-linked header and record coding account for the remaining 7.7 B. With a link in every record, even 12 records per frame cost 54.3 B each, more than the frame-linked format costs at $b = 4$. An exhaustive search over placement, batch sizes 1–12 and key-record interval returns row 8 as the byte-minimal configuration that meets the freshness, verifiability and MTU constraints, and row 4 for the record-linked format.

Joint Selection: The parameters are selected jointly because they are not independent. One signature per frame saves $g_a(1 - 1/b)$ per record. Delta coding saves $(1 - 1/b)$ of the difference between a key record and a delta record, since each frame must start with a key record. Both savings vanish at $b = 1$, where rows 1 and 5 cannot be improved. Batching is the choice on which the other two gains depend, and the deadline sets it.

Signature Scheme: Among the 64 B schemes, the signature scheme cannot change the bytes or the capacities of these rows: Ed25519 and ECDSA produce frames of

Table 5 Verifiability at frame loss $p = 0.05$ when one frame in r starts with a key record ($b = 4$, frame-linked format; 30 seeds, 95% intervals).

r	B/record	V measured	Eq. (3)	$V \geq 0.95$
1	43.25	0.952 [0.947, 0.957]	0.950	yes
2	41.38	0.921 [0.916, 0.927]	0.926	no
4	40.44	0.876 [0.863, 0.890]	0.881	no
16	39.73	0.655 [0.616, 0.693]	0.665	no

one size. They differ in verification time (Ed25519: 259 μs vs. ECDSA: 327 μs), which enters Eq. (4) only. BLS adds 32 B per frame and, as shown below, fails on CPU.

3.2 Loss and the key-record requirement

Delta coding usually places a key record every K records. With $K = 16$ and $b = 4$, only one frame in four decodes without its predecessor, and the frames that follow a lost one cannot be decoded until the next key record. Table 5 measures this with the codec in the loop: 7 200 frames per cell pass through the sender, an independent loss process and the receiver. Three things follow.

1. *Dependent Frames Miss the Target:* At the loss rate the system is specified for, $r = 4$ verifies 0.876 of records, not 0.95, and Eq. (3) predicts the measurement at every r. Under bursty loss a dependent design loses less — a Gilbert process with the same mean and bursts of four frames gives 0.929 at $r = 4$ — but it still does not reach $1 - p$.
2. *Limited Gain:* A cleaner link would admit a longer interval ($r = 4$ at $p = 0.02$, $r = 8$ at $p = 0.01$), but a longer interval can save at most 3.5 B per record.
3. *Specification Versus Measurement:* The loss rate here is a specification, not a measurement. At the loss of 2.3×10^{-4} we measured between two radios (Section 3.5), every interval in the table meets the target, $r = 16$ with $V = 0.998$.

AUTHBC is sized for the budget and therefore uses $r = 1$: each frame starts with a key record, costs 43.25 B per record and loses only its own records when it is lost.

Loss in the Simulator: Traced at capacity, 3 000 neighbor links in ns-3 lose frames independently of one another: bursts average 1.06 frames, against 1.055 for independent loss at the same rate. Eq. (3) is therefore the model of V at capacity, and the pessimistic one where losses are bursty. Fig. 3 shows AUTHBC under five loss processes with the same mean. With $\epsilon = 0.05$ and $p \leq \epsilon - 0.02$, it meets the target under every one of them, intervals included. At a looser budget the margin can be too small: at $\epsilon = 0.1$ and $p = 0.08$, the interval falls below the target under bursts of eight frames and under bit errors, although the mean does not.

Length-Dependent Loss: These values give every frame the same loss probability. At a fixed bit error rate, however, a longer frame is lost more often, and a batch is a longer frame. On a channel that loses 5% of the one-record frames of per-record signing, an AUTHBC frame is lost with probability 0.057. Then 0.943 of its records are expected to verify, below the target, and the measurement, 0.947 [0.941, 0.953],

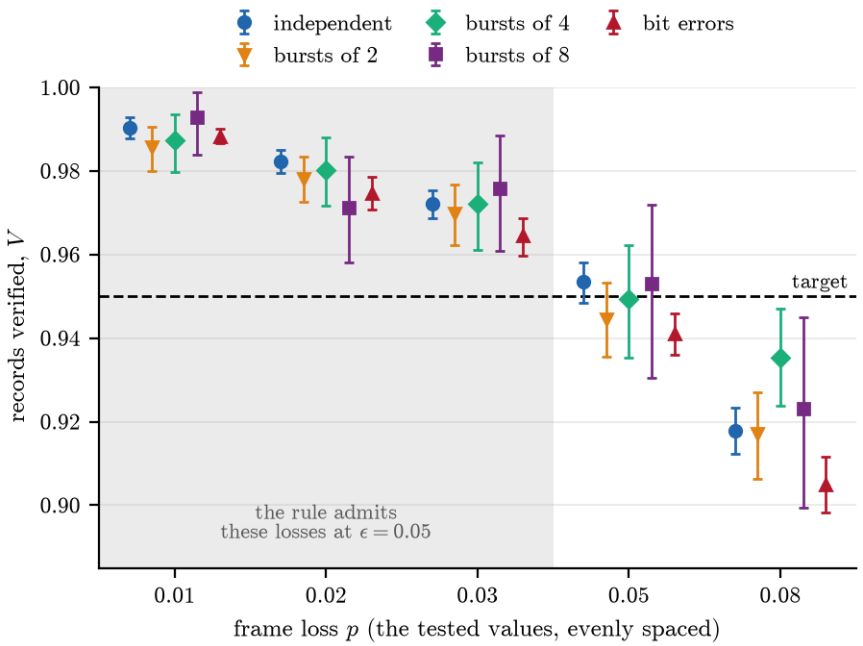


Fig. 3 Records verified by AUTHBC ($b = 4$, $r = 1$) against frame loss under five loss processes (95% intervals); for bit errors, p is one-record frame loss. At $\epsilon = 0.05$, every process meets the target, intervals included, while $p \leq \epsilon - 0.02$.

Table 6 Delay of the oldest record in a frame ($b = 4$, 50 records/s), with channel delay simulated at the heaviest load that still delivers 95% of frames.

	Fill (ms)	**Channel** (ms)	**Verify** (ms)	**Total** (ms)	**Margin** (ms)
Ed25519, mean channel delay	80	0.76	0.26	81.0	19.0
Ed25519, 99th percentile	80	2.40	0.26	82.7	17.3
Ed25519, worst observed	80	3.49	0.26	83.8	16.2
BLS, worst observed	80	3.49	8.61	92.1	7.9

agrees. AUTHBC reaches 0.95 only while one-record frames are lost at most 0.044 of the time. The capacities of Table 4 are simulated at equal, strong received power, where frames are lost by collision and not by bit error, and carry that qualification.

3.3 Freshness

Table 6 gives the terms of Eq. (2), with the frame sent as soon as it fills. The frame fills in 80 ms. Channel access and airtime add 0.76 ms on average and 3.49 ms at worst, and one Ed25519 verification adds 0.26 ms. The oldest record is therefore verified within 83.8 ms, leaving 16.2 ms of the 100 ms budget.

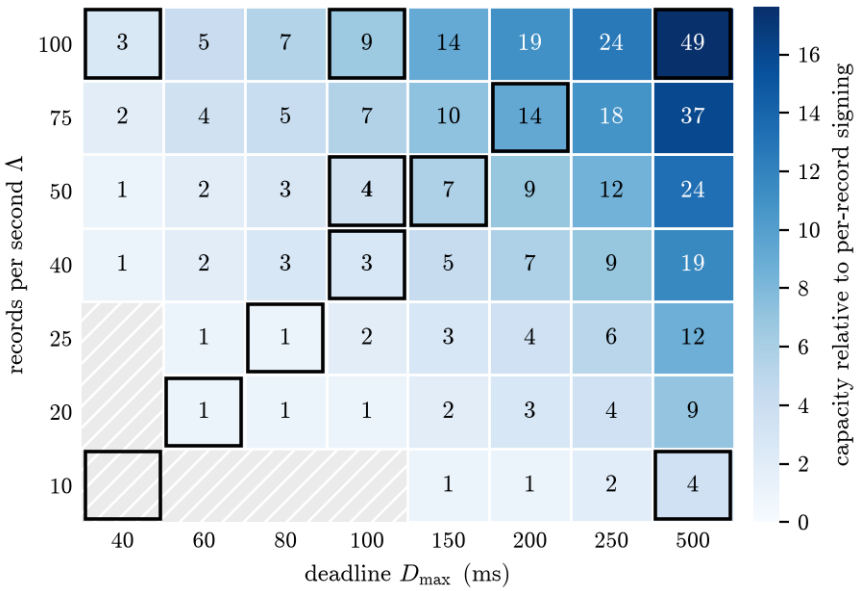


Fig. 4 Design map: the batch b^* each rate and deadline admit (in each cell) and its capacity relative to per-record signing (color). Outlined cells were simulated in ns-3; hatched cells admit none. Batching gains nothing unless $\Lambda D_{\max} > 2$.

Deadline Compliance: Batching b records requires a record rate above $b/D_{\max}$, here 40 records/s. At 20 records/s with the same 100 ms bound, the batch is one record and nothing is gained. Other rates and deadlines admit other batches (Fig. 4). At 15 combinations of rate and deadline, the simulated capacities lie within 1.05% of the contention model. At 100 records/s and 100 ms, where a batch of 9 is admitted, AUTHBC serves 6.63 times the nodes of per-record signing.

Age Under Loss: A lost frame adds one frame period, b/Λ, to the age of the next update. At $p = 0.05$ the mean peak age is 87.7 ms, within $D_{\max}$, but its 99th percentile is 169.8 ms. Thus 4.7% of updates exceed the deadline, about the same share that V loses.

3.4 Capacity

Bytes matter because they are airtime. Fig. 5 shows delivery falling as nodes are added, and Table 4 gives the resulting capacity for every row. Three things stand out.

1. *Capacity Gain:* In the frame-linked format, per-record signing holds 35 nodes at 95% delivery and AUTHBC 124 (3.5×). In the record-linked format the values are 32 and 88 (2.8×).
2. *Dominant Step:* The larger step is sending a quarter as many frames. Row 6, which shares the frame but not the signature, already reaches 76 (2.2× per-record signing); sharing the signature as well adds 1.6×. Each frame pays a preamble, an inter-frame space, a backoff and MAC framing whatever its length: about 173 μs,

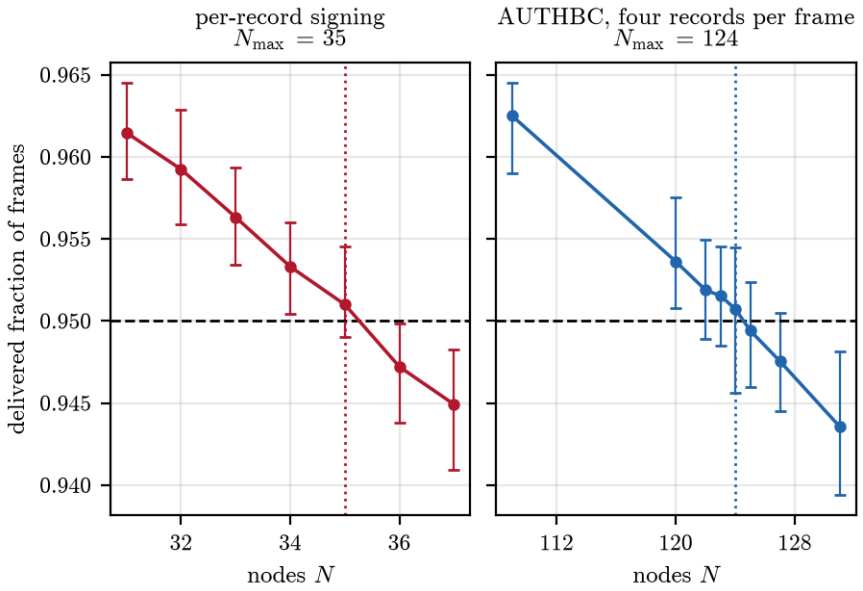


Fig. 5 Delivered fraction of frames against the number of nodes in ns-3 for per-record signing (row 5) and AUTHBC (row 8): mean and range of 30 runs. Delivery falls below 95% beyond 35 nodes for per-record signing and 124 for AUTHBC.

Table 7 Capacity $N_{\max}$ of per-record signing and AUTHBC at both operating points, in both formats (ns-3, 30 seeds, 95% intervals).

	Per-record	**AUTHBC**	**Ratio**
50 records/s, 100 ms			
record-linked	32 [32, 32]	88 [88, 89]	2.8×
frame-linked	35 [35, 35]	124 [124, 125]	3.5×
20 records/s, 250 ms			
record-linked	78 [77, 78]	219 [215, 221]	2.8×
frame-linked	85 [85, 86]	306 [306, 308]	3.6×

47% of what a one-record frame costs the channel. Removing three frames in four is therefore worth more than their bytes.

3. *Stability Across Operating Points:* Table 7 repeats the comparison at the relaxed operating point. The ratio hardly moves — 3.5× and 3.6× in the frame-linked format, 2.8× and 2.8× in the record-linked one — but it depends on the format, so we quote it with the format.

The dominant step is the principle of frame aggregation in IEEE 802.11n, whose aggregates are addressed to one receiver and exclude broadcast [8]. Batching at the application gives a broadcast sender the same effect.

Per-Run Capacity: These values require the mean over runs to reach the 0.95 target. Requiring that 95% of individual runs reach it gives 34 and 120 (31 and 86 in the record-linked format), at most four fewer nodes.

Capacity With a Loss Margin: These capacities sit at 5% frame loss, where V meets $1 - \epsilon$ with no margin. At 3% loss, which recommendation (5) in Section 4 requires for $\epsilon = 0.05$, ns-3 gives 27.6 nodes for per-record signing and 97.3 for AUTHBC (the crossings of 97% mean delivery), a ratio of 3.5. The contention model predicted both before the runs, within 0.6%.

3.5 Why capacity is where it is

The Airtime Capacity Law: At 5% loss, capacity follows a law in airtime. Let $f = \Lambda/b$ be the frames per second a node sends, T the time one frame holds the medium (its airtime plus DIFS, the DCF inter-frame space), c twice the preamble-detection time of the simulator, and a one fitted constant. Then

$$N_{\max} \approx \frac{0.05}{f\,(a\,T + c)}, \qquad a = 0.071, \quad c = 8\,\mu\text{s}. \tag{6}$$

We call Eq. (6) the airtime capacity law; it is the capacity envelope of the title. It was fitted to six rows of Table 4 (1, 2, 4, 5, 6 and 8), which it then reproduces within 3.8%. Twelve further configurations were predicted from the law before they were simulated. Seven of our own all lie within 3.4% of it. The other five, the one-record frames of the stream-signing schemes in Table 15, lie 3.5–4.2% above the law's prediction. A single utilization ceiling, measured at one point and applied to all, misses the first thirteen (six fitted, seven predicted) by up to 15%, in both directions. Equation (6) is a fit at one loss level and one PHY rate, checked for frames of 94–565 B, 5–50.5 frames/s and 28–306 nodes; the tables quote simulated values, not the law.

All of those configurations hold one record or four. Batches of two, three and eight, which deadlines of 60, 80 and 180 ms would admit, were predicted before they were simulated. Their capacities are 66.7, 96.5 and 222.3 nodes, within 1.6% of the law and 0.8% of the contention model. Fig. 6 places them on the law: it shows what a deadline buys at this record rate, and at 40 ms it buys nothing.

The Contention Model: The constant a is not free. A frame is lost in two ways. Two stations that found the medium busy may draw the same backoff slot, a mechanism known from analyses of vehicular safety broadcast [40, 41]. Or a station decides to send within the 4 μs in which it cannot yet sense another; that is c. This is the vulnerable period of carrier sensing [42], with the detection time in place of the propagation delay; the DCF broadcast analyses above omit it. We call an event simulation of the DCF access procedure with these two mechanisms, with nothing fitted and none of the simulator's code, the *contention model*. It reproduces all 18 simulated capacities within 1.2%, and it predicted all six cases that had not been simulated — the contention window doubled, and loss levels of 2% and 10% — within 1.7% (Appendix B).

Models Against the Simulator: The broadcast model [28] matches saturated ns-3 runs within ±0.51% over $N = 5$–50 (30 seeds).

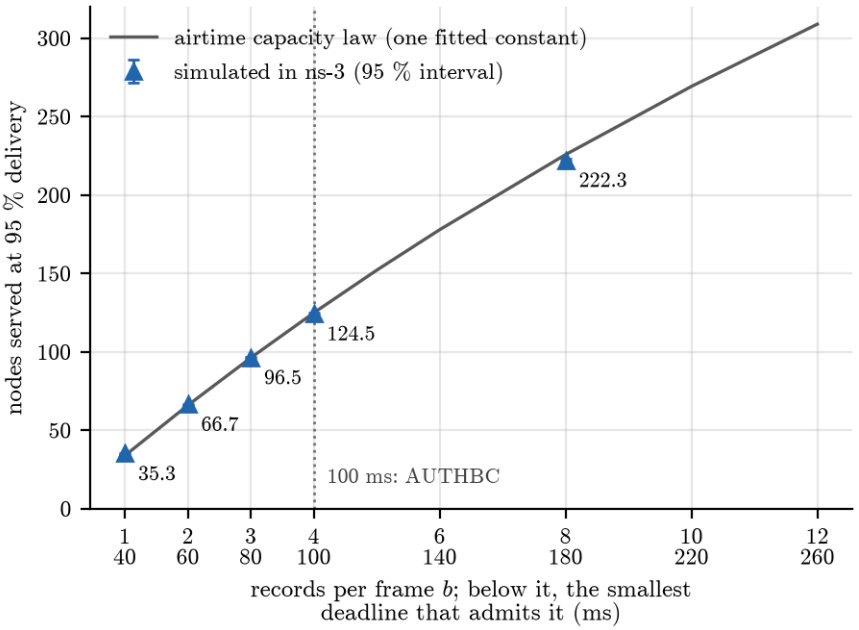


Fig. 6 Nodes AUTHBC serves at 50 records/s, by batch size and the smallest deadline that admits it: simulated capacities and the airtime capacity law. At 40 ms the batch is one record and nothing is gained; longer deadlines buy capacity as the law predicts.

Contention Model Against Two Radios: On two Raspberry Pi 4 radios, one transmitter alone loses 4 frames in 17 600 ($p = 2.3 \times 10^{-4}$). With both radios saturated, 12.4% of the frames put on air are lost where the contention model gives 11.8%, as predicted before the run. Below saturation the agreement is poorer. As the medium fills from half to 85%, the radios lose 1.13, 0.79 and 0.56 times what the model predicts; the prediction made before the run held at the first two loads and failed at the third. The cause is not established. Below saturation a radio often holds two frames of its own, and a radio of this family sends its queued frames closer together than the standard allows: one per 2.019 ms against 2.078 ms, as commercial cards are known to [43]. That state is common with two busy radios and rare at the operating point. However, a follow-up that switched off the vendor's frame bursting failed all four of its predictions: bursting has no demonstrated effect on the loss. No capacity was measured: the capacities in this paper rest on simulation, and they are the capacities of radios that keep the DCF access procedure.

3.6 Receiver CPU

Equation (4) charges cryptography only. By it, one core of the Raspberry Pi 4 serves a neighborhood of 296 nodes when each frame carries one Ed25519 signature, and of 77 when every record is signed. At its capacity AUTHBC uses 42% of a core, so by this count the channel binds first. Row 6 shows that the count can bind: with a signature on every record, it reaches the limit of the channel at 98% of a core. With

Table 8 Time to receive a four-record frame on one Raspberry Pi 4 core, and the nodes one core serves by Eq. (4), with the receive time in place of its cryptographic terms. All four receivers were timed in one session.

Receiver	μs per frame	Nodes per core
Python	1 409	57
Rust	277	290
Rust, batches of 64	124.4	643
Rust, deferred[1]	10.2 + 117.2	629

[1]The group MAC is checked first, then the frame is decoded (first time); the signatures are verified later, in batches of 64 (second time). Both receivers check the group MAC before decoding.

BLS a verification takes 8.61 ms and one core serves 10 nodes. On this hardware BLS is therefore excluded by CPU long before the channel, in addition to its larger signature. Ed25519 was designed for batch verification [35]: on the Pi 4, a batch of 64 takes 0.46 of the time per signature. The Python prototype verifies one signature at a time, and ECDSA does not admit batches.

Decoding, not cryptography, limits the full Python receiver. A four-record frame takes 1.43 ms to receive on the Pi 4, 5.5 times its verification, so one core serves 57 nodes and the processor binds before the channel. Run as one receiver process per core, one to four at once, the board serves 56, 106, 150 and 187 nodes. Four cores receive 3.36 times the frames of one, less than we had written down beforehand, so 124 nodes need three of them. In Rust, the same receive path is 5.1 times faster, and the channel binds again (Table 8). A receiver can also use records as soon as their group MAC and frame are checked, and verify their signatures in batches before they enter the ledger. This deferred batch verification serves 629 nodes per core. A record is then evidence within 191 ms by the convention of Eq. (2); within that window, a member holding the group key could have records acted on that are later purged. The Python receiver was timed in three sessions on the same board, at 1.41–1.44 ms per frame (within 2%); each table and paragraph quotes the session it reports, and Section 3.10 adds the admission layer.

3.7 Certificates

The rows so far assume that receivers already hold the public keys of the senders. Vehicular practice sends a full certificate in every fifth message and an eight-octet digest in the others [10]. Charging that to every frame — a 162 B certificate, so 38.8 B per frame on average — gives the first certificate column of Table 4 (per-record signing: 184.57 B vs. AUTHBC: 52.95 B per record). That reading favors AUTHBC. The practice it copies sends ten messages a second, so one in five is one certificate per 500 ms; per-record signing at 50 frames/s, charged per frame, sends five times that. The second column therefore charges one certificate per 500 ms at any frame rate (per-record signing: 159.93 B vs. AUTHBC: 51.41 B), and batching then amortizes only the

Table 9 Sender energy per record on a Raspberry Pi 4 (μJ): metered median and range of five repetitions, against the model of time multiplied by power.

	Metered	Model	Gap
record-linked, every record signed	118.8 [118.5, 119.9]	105.8	12.3%
record-linked, one signature per 4	58.4 [58.1, 59.3]	54.3	7.5%
frame-linked, every record signed*	273.1 [270.6, 276.9]	284	−3.9%
frame-linked, one signature per 4*	155.4 [155.1, 155.8]	163	−4.4%
frame-linked, Rust, every record*	60.7 [60.2, 62.4]	70.2	−13.6%
frame-linked, Rust, one per 4*	17.1 [16.9, 17.1]	19.4	−11.6%

* Frame-linked rows: second board and sensor.

digest. These are explicit certificates. Implicit ones are shorter (a signed message of at most 226 B against 330 B [10]) and are not priced, so both columns are upper bounds.

3.8 Sender energy

Table 9 gives the metered sender energy in both formats. It shows three things:

1. *Batching Saves Energy:* In the record-linked format, encoding, hashing and signing a record costs 58.4 μJ with batching against 118.8 μJ when every record is signed, 51% less. The frame-linked sender in Python meters 155.4 μJ per record against 273.1 without batching, 43% less.
2. *Model Agreement:* The model of the record-linked format composes separately measured times with one power, the median over these same metered runs, so the comparison tests the times: the model is 7.5–12.3% low, the gap being frame assembly in the Python prototype. The frame-linked sender, whose model is its timed frame (0.87 ms for four records) at that power, reads −4.4% and −3.9% against its model. Its rows were metered on a second board and sensor, where a control repeating the first row gives 116.2 μJ.
3. *Bytes Versus Energy:* In Python, the order between the formats is the reverse of the bytes: the frame-linked sender costs 2.7 times the energy per record of the record-linked one, because its codec, written for the size of the frame, is, in this prototype, the slower to run. In Rust it costs 17.1 μJ per record, a factor of 3.4 lower than the record-linked format with batching: the order follows the bytes, and the reversal was Python's. Its model reads −11.6% and −13.6%, because Rust code draws 0.66 W above idle, less than the one power of the model. A control repeating the first row that day gives 118.0 μJ.

Receiver and radio energy are modeled, not metered, and for the record-linked format only, the radio's power coming from a unicast measurement on another band. Adding one receiver and the radio, the model gives 131.7 μJ per record with one signature per four records, against 376.8 μJ when every record is signed.

3.9 Robustness

The results above assume one collision domain of static nodes at equal received power, one PHY rate, Ed25519 and synthetic telemetry. This section varies each.

Unequal Received Power and Capture: All results so far place every node at one point, with equal received power, so a frame that overlaps another is lost everywhere. With the nodes placed uniformly in a disc and power falling as in free space, a receiver keeps the strongest of the frames that start together if it is 4 dB above the rest. The contention model with that one rule, read in the source of the simulator, gave 39.5 and 139.7 nodes for per-record signing and AUTHBC before the runs. In a disc of radius 15 m, where every station still senses every other by energy, ns-3 gives 39.6 and 140.4. At 100 m, the scale of a swarm, it gives 39.4 and 139.3, at most 0.7% lower. The equal-power capacities of Table 4 are therefore conservative for unequal power, by 12–13%, and the capacity ratio stays 3.5×. Both discs keep every pair in range: hidden stations, which would lower the capacity, are not in this test.

A Spread, Moving Swarm: Hidden stations appear when a swarm is spread wider than one sensing range. We placed the nodes in discs of radius 100 m to 1 km, with log-distance path loss, Nakagami fading and the same capture rule, and also in a grouped and a moving layout. Across the spread and moving layouts, the capacity ratio of AUTHBC to per-record signing stays between 3.46 and 3.63 (Fig. 7). At a radius of 1 km, 11% of the interferers of a frame cannot hear its sender. Spread over that disc, the channel carries more nodes, 218 with AUTHBC and 63 with per-record signing, but each node has fewer neighbors within reach: 8.7 and 2.5, against 124 and 35 nodes in one collision domain, because a receiver contends with every node in its carrier-sensing range [44]. A node's neighbors within reach are the other nodes within 215 m of it, the largest distance at which an isolated link delivers at least 99% of frames; they are counted for every frame sent and read at the 95% delivery crossing. With every node on its own Gauss–Markov path at up to 15 m/s [45], delivery changes by at most 0.0005. The channel model has no Doppler and no time-correlated fading.

Choosing the Batch at Run Time: A controller that sets b from the channel occupancy it measures was simulated on a swarm that grows while it flies. As specified before the runs, with a margin of 0.01, it met $V \geq 0.95$ in 75% of one-second windows and failed at 76 nodes. With a margin of 0.02 it met the target in 93% of windows and failed at 116, against 99% and 127 with $b = 4$ fixed, while saving 21 ms of peak age (Fig. 8). These ages come from the randomized source of the capacity runs, which sends each frame at a random instant of the next period. They are therefore on average half a frame period longer than the send-at-fill ages of Section 3.3, and they are not comparable with $D_{\max}$. Choosing b at run time buys freshness at the cost of reliability margin; it does not raise capacity. The nearest published mechanism adapts the contention window to a flocking swarm [46].

Other PHY Rates and a Post-Quantum Signature: The capacities were also simulated on 802.11a at 12 and 24 Mb/s and on the 10 MHz channel of 802.11p at 3, 6 and 12 Mb/s (Table 10). AUTHBC serves 3.39–3.80 times the nodes of per-record signing, more at faster rates: a faster PHY shortens the payload but not the fixed cost of a frame. The contention model stays within 1.5%, except at the fastest rate of each channel width, where it is 2.9–4.6% optimistic. Falcon-512, with its padded

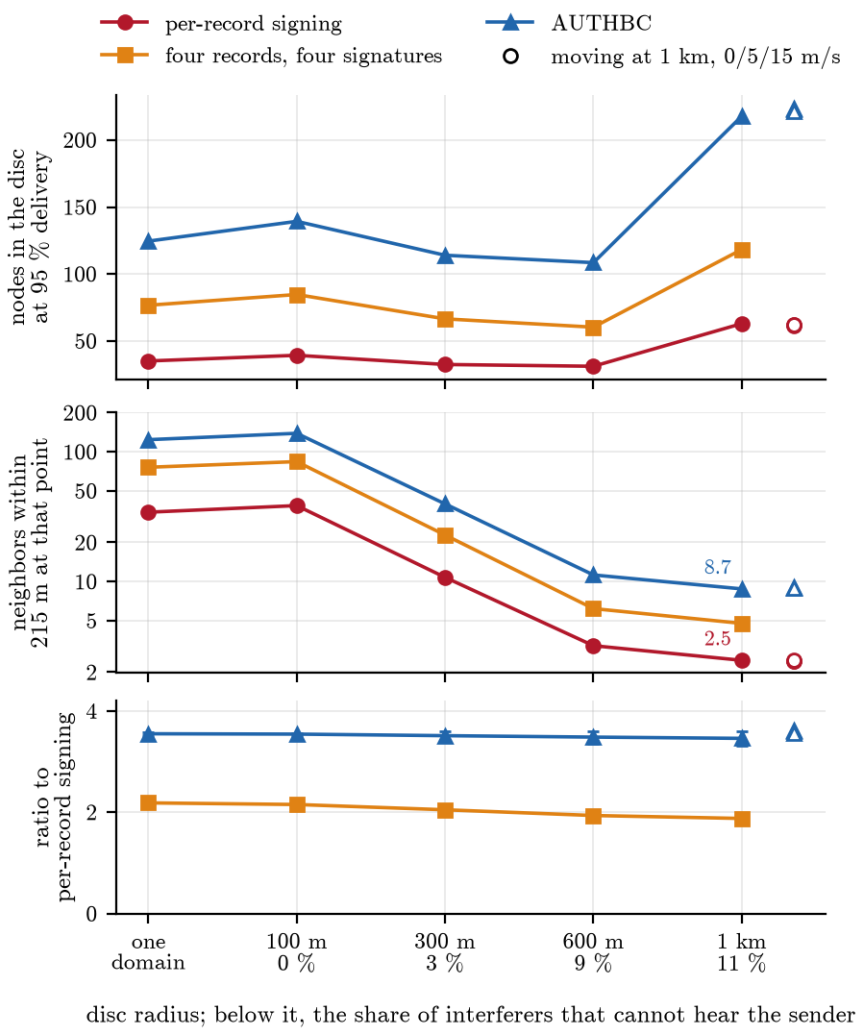


Fig. 7 As the swarm spreads: nodes in the disc at 95% delivery (top), neighbors within 215 m of a receiver there (middle, log scale), and AUTHBC's ratio to per-record signing (bottom). Hollow markers: moving at 0, 5 and 15 m/s. Neighbors in reach fall; the ratio holds.

signature of 666 B, fits a frame. Simulated in the same way, per-record signing serves 12.4 nodes and AUTHBC 43.5, 3.5 times as many, as with Ed25519: the gain comes from sending a quarter as many frames, whatever the size of the signature. Its frames, of 749 and 776 B, lie outside the range on which the airtime capacity law was checked, so they were checked against the contention model. It predicted 12.3 and 43.2 nodes, within 0.6% of the simulated capacities; the law misses them by up to 7%. On the Pi 4, Falcon-512 signs in 638 μs and verifies in 88 μs, faster than Ed25519 verifies.

Representativeness of the Synthetic Telemetry: Every size above comes from one generator, whose delta record is exactly 9.0 B in every sample, the floor of the format.

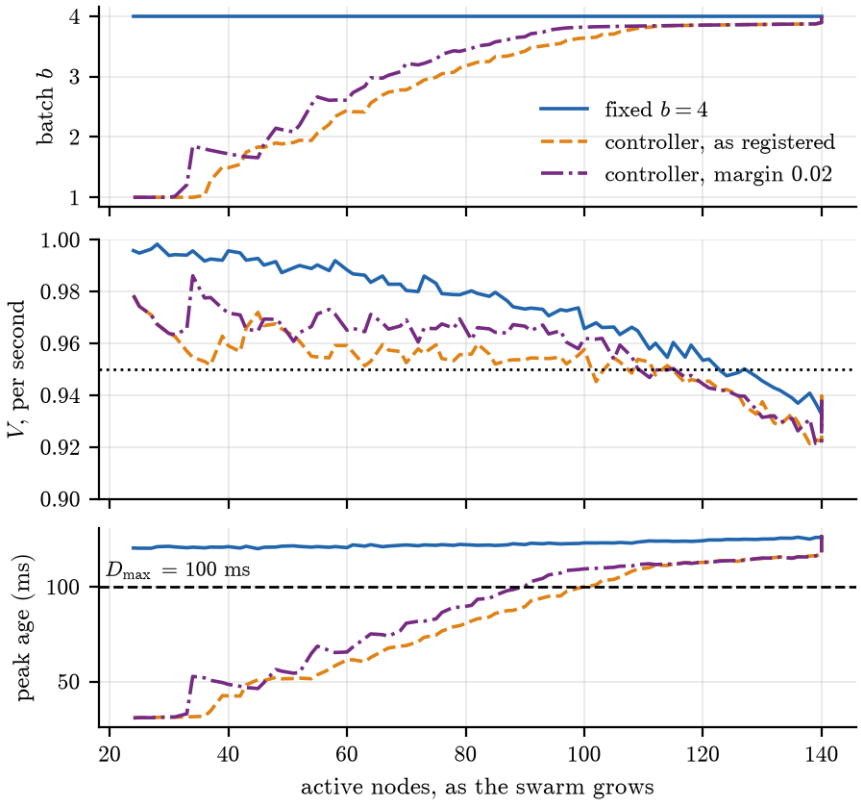


Fig. 8 A growing swarm, mean of ten seeds: the batch each policy chooses, records verified per second, and peak age (dashed: $D_{\max}$). Ages include the random send instant of the capacity source, half a frame period above send-at-fill. Run-time batching buys freshness, not capacity.

Twelve public PX4 flight logs (four quadrotors, two hexa/octorotors, three fixed wings and three VTOLs; 29 092 in-flight records), chosen by a rule fixed before any was opened, were passed through the same encoder. The logs carry position at 5 Hz, so the comparison is at 0.2 s spacing and above, not at the 50 Hz of the operating point. At 0.2 s a real delta record averages 11.1 B against 10.8 B for the generator, and a real key record 22.0 B against 24.0. The spread follows speed: the six multicopters, mostly hovering, average 10.0–10.5 B, and the three fixed wings up to 13.2 B. With real sizes at 0.2 s AUTHBC saves 69.1% (68.2% in the least favorable log), where the generator gives 69.4%. The generator is representative to within about one percentage point, and optimistic for fast vehicles.

For the operating rate itself we flew PX4 in software-in-the-loop simulation — its own estimator, logger and MAVLink module on a simulated quadrotor — with position logged every 20 ms. Over 14 317 in-flight records at up to 10 m/s, every delta record is 9.0 B; the key record averages 20.9 B, and AUTHBC costs 42.47 B per record, 1.8% less than with the generator's records. The 50 Hz position stream of the same autopilot holds a 20.0 ms period to within ±0.4 ms (central 68%). 30 further simulated

Table 10 Capacity in ns-3 on other PHYs and with a post-quantum signature, and the error of the contention model (negative values: the model is optimistic).

PHY and rate	Frame (μs)	$N_{\max}$ per rec.	$N_{\max}$ AUTHBC	Ratio	Model (%)
802.11a, 6 Mb/s	338	35.3	124.7	3.54	+0.5
802.11a, 12 Mb/s	198	50.5	184.2	3.65	−1.5
802.11a, 24 Mb/s	126	63.0	239.7	3.80	−4.6
802.11p, 3 Mb/s	666	20.4	69.1	3.39	+0.4
802.11p, 6 Mb/s	386	30.7	109.6	3.58	−1.1
802.11p, 12 Mb/s	242	41.2	151.6	3.68	−4.0
Falcon-512, 802.11a 6 Mb/s	1 142	12.4	43.5	3.5	+0.6

Frame: AUTHBC's, on air. Model: the larger gap of the two rows. The 802.11a 6 Mb/s row reads the runs of Table 4 on this study's grid of 3-node steps.

Table 11 Record sizes in bytes from 30 simulated flights, ten per airframe, at 50 records/s, and what AUTHBC saves.

Fields	Key	Delta quad	wing	VTOL	Saved	Ratio
9	22.8	9.0	9.2	9.2	70.1%	3.51
15	32.8	16.0	16.2	16.2	67.2%	3.31
25	45.5	26.0	26.3	26.2	63.3%	3.04

Saved: bytes on air. Ratio: capacity $N_{\max}$ of AUTHBC over that of per-record signing, by the contention model.

flights of the same autopilot, ten each of a quadrotor, a fixed wing and a VTOL, the fixed wings at their planned 15–25 m/s, give records of 9, 15 and 25 fields at the 20 ms spacing of the operating point (Table 11). Almost every field changes by less than its first byte can hold, so a delta costs about one byte per field on every airframe; a fixed wing costs 0.2 B more than the quadrotor. With 15 fields, adding attitude and body rates, AUTHBC saves 67.2% of the bytes and serves 3.31 times the nodes. With 25 fields, adding navigation quality, airspeed, wind, mission state and thrust, it saves 63.3% and serves 3.04 times the nodes. The gain is set by the number of frames, not by the size of the record.

3.10 Security evaluation

Table 12 lists the evidence for each property: the Tamarin proofs of Section 2.1, and the measurements below.

Ledger Audit: One pass over the frames of a mission, with the public keys of the senders, finds every equivocation and every frame that does not verify. For three UAVs over a flight hour, a planted equivocation and a planted forgery were both found, with

Table 12 Security properties and threats, with the mechanism and the evidence for each: Tamarin proofs, simulated clocks and floods against one Raspberry Pi 4 receiver core.

Property or threat	Mechanism	Evidence	Result
(i) Origin and integrity	signature over the frame	Tamarin, model without chain and clock	proven
(ii) Replay	increasing sequence numbers	same model	proven
(i) and (ii) together	both	same model	injective agreement proven
(iii) Equivocation	both frames kept	same model	proven: it implies a revealed key
Gap in a sender's frames	sequence numbers	same model	detection proven
(iv) Continuity	chain link per frame	Tamarin, model with the chain	proven
(v) Delayed frame	timestamp window Δ	Tamarin, model with the clock	accepted without the window, refused with it
		simulated clocks, 72 000 honest frames per setting	with $\Delta = 150$ ms no honest frame refused at clock errors up to 10 ms; every frame held 250 ms or more refused; a 250 ms window outlasts 75 min without GNSS
Flooding by outsiders	8 B group MAC	UDP flood, 41 senders on one core	$V = 0.984$ at 3 000 forged frames/s (Fig. 9); the MAC costs 3.1% of capacity
Flooding by a member	none	same	$V < 0.95$ from 129 forged frames/s
Ledger audit	signed frames uploaded	three UAVs, one flight hour, 391 495 frames	both planted faults found, no false finding, in 6.4 min of one core

no false finding, in 6.4 minutes of one core. The audit covers the 135 000 frames they sent and the 256 495 verified copies they kept of each other's frames over links losing 5%, 391 495 frames in all. Keeping the frames costs 42.9 B per record. That is slightly less than the 43.25 B of Table 4: the mission's sender identifiers and sequence numbers are smaller than the table's sizing convention assumes and take fewer CBOR bytes, which outweighs the longer key records later in the hour.

Delayed Frames: The timestamp acceptance window refuses a frame whose newest record is more than Δ older than the receiver's clock. With $\Delta = 150$ ms, in 72 000 honest frames per setting, no frame was refused at clock errors up to 10 ms, and every frame held back by 250 ms or more was refused. In a GNSS outage, two clocks drift apart by 144 ms per hour. A fixed window of 250 ms then holds for 75 minutes, and a window widened by that drift keeps working, its bound growing at the same rate. The window is held against a frame's newest record, taken as the frame leaves, so an honest frame arrives 3.75 ms old (channel and one verification), and 246 ms of a 250 ms window remain for clock drift and the errors of the two clocks (14 ms apart, one standard deviation). After 75 min without GNSS the drift is 180 ms and the margin

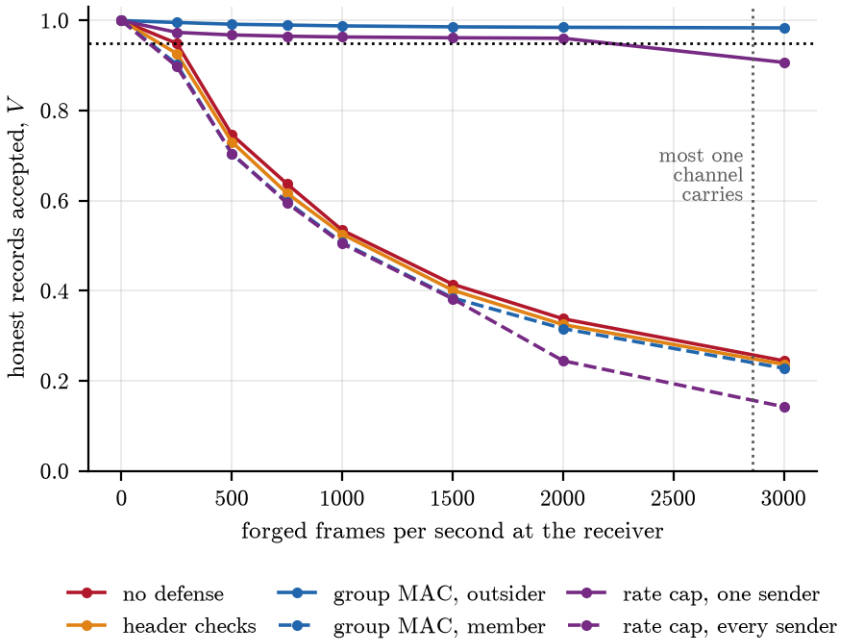


Fig. 9 Honest delivery on one Raspberry Pi 4 receiver core with 41 senders, against the rate of forged frames, for each defense and attacker. The group MAC stops outsiders; a member holding the key and forging in every sender's name is stopped by neither defense.

66 ms, 4.7 standard deviations: one of 72 000 honest frames was refused. After 90 min (216 ms of drift), 1.6% were.

Flooding: A forged frame — with a real sender, a newer sequence number and a well-formed signature under another key — costs the Python receiver 1.08 ms on the Pi 4, against 1.44 ms for an honest one, which also pays for its storage. The flood uses 41 senders, one core's share of the 124 nodes for which the Python receiver needs three cores (Section 3.6). Unattacked, they take 80% of the core and every honest frame is accepted, whereas all 124 senders on one core leave $V = 0.43$. With 41 senders on one receiver core, 247 such frames per second push honest delivery below 0.95 (Fig. 9). In the Python receiver, the 8 B group MAC is checked before the frame is decoded and refuses an outsider's frame in 22.5 μs. At 3 000 forged frames per second, more than the 2 857 that one channel carries, honest delivery stays at 0.984. A member holding the key is stopped neither by the group MAC nor by a per-sender rate cap: honest delivery falls below 0.95 from 129 forged frames per second behind the MAC, and from 123 behind the cap when the forgeries name every sender. Against one forging sender, the cap keeps delivery at 0.96 or more, up to the rate one channel carries. The group MAC costs 3.1% of capacity.

3.11 Low-rate links: where no batch helps

LoRaWAN limits the payload of a frame by data rate, and its EU863-870 plan defines twelve data rates [47]: seven LoRa (DR0–DR6), one FSK (DR7) and four LR-FHSS

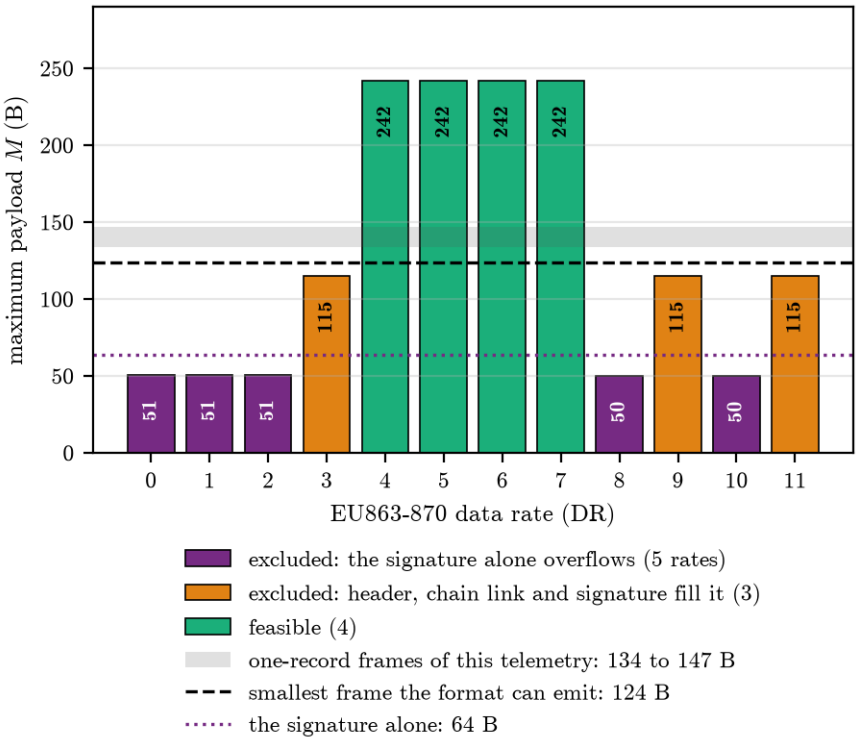


Fig. 10 Payload limit of each EU863-870 data rate against the smallest frame-linked frame and the one-record frames of this telemetry, with a 64 B signature. Eight of the twelve data rates cannot carry one frame.

Table 13 Fit of one self-verifying frame within the EU863-870 payload limits, by authenticator (frame-linked format).

	Floor† (B)	**Range**‡ (B)	**50/51 B** 5 rates	**115 B** 3 rates	**222/242 B** 4 rates
64 B signature, chain link	124	134–147	no	no	yes
… link not sent on air	89	99–112	no	yes	yes
48 B BLS signature	108	118–131	no	no*	yes
13 B symmetric tag	72	82–95	no	yes	yes

† every field zero. ‡ frames of this telemetry.
* a frame of zeros would fit; no frame of this telemetry does.

(DR8–DR11). By Eq. (5), a self-verifying frame needs a header, a chain link, a signature and one key record. The smallest such frame that the frame-linked format can produce at all — sender 0, sequence 0, every field zero — is $16 + 35 + 64 + 9 = 124$ B; this frame was produced, signed and verified. Header, link and signature alone occupy 115 B. The header of this floor frame is 16 B, one byte less than in Table 2: its record

stream is a single 9 B record, and CBOR writes the length of a byte string shorter than 24 B in its type byte.

Fig. 10 compares that floor with the payload limits. Three things follow:

1. *Signature-Bound Rates:* Five data rates (DR0–DR2, DR8 and DR10) allow 50 or 51 B; the signature alone does not fit.
2. *Format-Bound Rates:* Three data rates (DR3, DR9 and DR11) allow 115 B; header, link and signature fill the payload, and no record fits.
3. *Feasible Rates:* Four data rates (DR4–DR7) allow 222 or 242 B and can carry a frame.

Eight of twelve data rates cannot carry one signed, chained frame, in either format and under either of the payload tables of the standard. This exclusion bound does not depend on the telemetry, since each rate is excluded against the floor of the format; three of the eight depend on the format (condition v below).

Scope: The result holds under five conditions. (i) The limits are the regional parameters of LoRaWAN; the LoRa PHY itself carries 255 B [48] and is not excluded. (ii) Every frame carries a public-key signature. (iii) There is no loss recovery. (iv) $\epsilon \leq p$, which by Eq. (3) forbids spreading a signature over several frames; at $p = 0.01$ and $\epsilon = 0.05$ a signed unit may span five frames, and the exclusion no longer holds as stated. (v) The frame is the format of Section 2.2. The five rates of 50 and 51 B are excluded by the signature alone, whatever the format. At 115 B a signature and a 32 B hash leave 19 B, which the header and framing of this format fill exactly. A hand-packed layout would leave part of them for a record, but less than the 24.0 B that our key record averages.

Effect of Each Relaxation: Table 13 quantifies what each relaxation buys. Taking the chain link off the air frees the three 115 B rates: they are excluded by the link, not by the signature. The smallest standardized signature, 48 B BLS12-381 [49], fits a 51 B payload with 3 B to spare. All BLS timings in this paper are of the 96 B variant of blspy; the 48 B variant was not timed. It verifies with the same two pairings, and to leave the channel as the binding limit at 124 nodes it would have to verify in 0.64 ms, 13 times faster than the 8.61 ms measured. At 115 B, a frame whose fields are all zero would fit (108 B), but the smallest frame of our telemetry is 118 B, a three-byte shortfall that is not counted as an exclusion. A 13 B symmetric tag, as in MAVLink 2, fits the 115 B rates. At 50 or 51 B, however, header and link already take 51–58 B, so a hash-chained frame does not fit there under any authenticator.

What the Feasible Rates Carry: At DR5 under the 1% duty cycle, a 242 B frame holds 8 records and sustains 0.20 records/s per node (5 records and 0.13 in the record-linked format). The batch must be sized at the record spacing it implies. At this rate the records of a frame are 5.0 s apart (6.1 s in the simulated frame below); their delta records were sized at the nearest spacing measured, 5.5 s, where one costs 13.6 B, not the 9.0 B it costs at 50 ms. The ns-3 LoRaWAN module carries at most 222 B, where a frame holds six records. Simulated with a 218 B payload every 36.4 s (0.165 records/s per node), on one channel with one demodulator and no capture, the largest neighborhood with mean $V \geq 0.95$ is three nodes (30 seeds, 95% interval $[2, 3]$; one if 95% of runs must each reach it). The closed-form model of Haxhibeqiri *et al.* [30], fitted

Table 14 Bytes per message of certificateless aggregate schemes as reported in [7] (67 B message), and per record of AUTHBC with certificates charged per frame.

Scheme	Bytes per Message
Wang *et al.**	859
Liang *et al.**	735
Xu *et al.**	596
Cahyadi *et al.**	583
Li *et al.* [7]	583
per-record signing (row 5), with certificates	184.57
AUTHBC (row 8), with certificates	52.95

* total per message as reported in [7].

Table 15 Stream-signing schemes at 50 records/s, simulated as in Table 4. "In time" counts a record only if it verifies within $D_{\max}$.

Scheme	Auth. B/rec.	Total B/rec.	N_{sat}	$N_{\max}$	In time
MAVLink 2 tag[1]	13	93.8	23	42.1 [42.0, 42.3]	42.1
TESLA[1,2]	24	105.8	22	40.4 [40.2, 40.7]	39.6–39.8[3]
Gennaro–Rohatgi[2]	32	113.8	21	39.4 [39.3, 39.6]	none
EMSS[2]	65	147.3	20	34.8 [34.7, 35.0]	none
Wong–Lam, block of 4	129	210.8	17	29.0 [28.8, 29.1]	29.1
per-record signing	64	145.8	20	35.3 [35.1, 35.4]	35.3
AUTHBC	16	43.3	52	124.5 [124.0, 125.0]	124.5

$N_{\max}$ and in time: crossings of 0.95, interpolated between the two simulated sizes around it (Section 3.12).

[1] No non-repudiation.

[2] A packet does not verify alone.

[3] Disclosure delay of two intervals of at most 20 ms.

to hardware interference measurements and evaluated at the same operating point, gives four. Such a neighborhood carries about 0.5 records/s in all, where AUTHBC on 802.11 carries some 6 200. On these links the ledger therefore records provenance every few seconds; it is not the 802.11 design at a smaller scale.

3.12 Comparison with published schemes

Stream Signing: Table 15 places the schemes of Section 1.1 at our operating point. Each was implemented and run in ns-3 in the frame-linked one-record frame, with its own authenticator in place of the signature. All of them send a frame per record — 50.0 frames/s per node (50.5 for EMSS, with its signature packets), where AUTHBC sends 12.5 — so the fixed channel time of a frame is paid for every record, whatever the size of the authenticator. A 13 B tag in place of a 64 B signature raises $N_{\max}$ from 35.3 to 42.1 nodes but gives up non-repudiation; four records in a frame raise it to 124.5.

When a record counts only if it verifies within the 100 ms deadline, only MAVLink 2, Wong–Lam and TESLA with a short disclosure delay reach 95%. Gennaro–Rohatgi and every variant of EMSS never do, because a record waits for packets sent after it. Tree chaining is the one scheme that keeps what a ledger requires — every packet verifies alone, and its origin can be shown to a third party — and it pays 129 B per packet for it. These schemes amortize the cost of signing; on a contended channel, the cost that binds is the frame. Both capacity columns of Table 15 are crossings of 0.95, interpolated between the two simulated sizes around it. In-time delivery is counted on the 20 sender–receiver pairs traced in each run and delivery on all pairs, so the in-time crossing of Wong–Lam (29.1) lies above its delivery crossing (29.0) by sampling alone, inside the interval $[28.8, 29.1]$.

Aggregate Signatures: The nearest published comparators are certificateless aggregate signature schemes for vehicular broadcast. Table 14 lists what one signer sends per message; for n messages it is n times as much, since each carries its own identity, key and signature components. Aggregation acts afterwards, at a roadside unit: it reduces the work of the verifier and what the unit forwards — in the one scheme read at source, 388 B for a single signature and 784 B for the aggregate of a hundred [50]. Batching acts before, on the own records of a sender. The two act on different links, and the table should not be read as a ranking: those schemes provide conditional privacy, which AUTHBC does not; their group elements are 128 B where our signature is 64 B; and their values are for a different message. What the comparison does show is that per-message authentication of this kind costs several hundred bytes, and that aggregation alone does not change that.

4 Discussion

4.1 Configuration selection

Table 16 maps each deployment scenario to the configuration that the constraints of Section 2.3 admit, and to the constraint that binds.

4.2 What generalizes

One insight carries beyond this paper: on a contended broadcast channel the cost that binds is the frame, not the authenticator. A frame costs about 173 μs of channel time whatever its length, so schemes that amortize signing but send a frame per record gain little, and a post-quantum signature leaves the gain of batching unchanged.

The same accounting applies to other broadcast systems (Table 17). ADS-B 1090ES and Bluetooth 4 Remote ID cannot carry a signature at all. An ADS-B message carries 7 B of data [23] and a Bluetooth 4 Remote ID message 25 B; a message pack over Bluetooth 5 or Wi-Fi holds up to nine such messages, 225 B [22]. Bluetooth 5 and Wi-Fi Remote ID fit up to seven records in a frame, but at one message a second no frame meets any deadline up to 1 s. V2X CAM at 10 Hz admits no frame at 100 ms, one record at 200 ms and two at 300 ms. This is consistent with those standards signing each message, although it is not offered as their reason.

Table 16 Configuration selection by deployment scenario, with the constraint that binds.

Scenario	Signing	b	Binds	Outcome
802.11, $\Lambda D_{\max} = 5$	per frame	4	freshness	43.25 B/record[1]
802.11, $1 < \Lambda D_{\max} \leq 2$	per record	1	freshness	no batch admitted
802.11, $\Lambda D_{\max} \leq 1$	–	–	freshness	no frame meets $D_{\max}$[2]
LoRaWAN, DR4–DR7	per frame	8[3]	duty cycle	0.20 records/s
LoRaWAN, eight rates	none	–	payload	no frame fits[4]
Python receiver	per frame	4	CPU	57 nodes per core
Rust receiver	per frame	4	channel	290 nodes per core
BLS12-381	not advised	–	CPU	10 nodes per core

[1] $N_{\max} = 124$ at 5% frame loss and 97.3 at 3%, in simulation; no capacity was measured.

[2] Under the peak-age convention of Eq. (2), which counts the whole sampling interval; with the wait $(b-1)/\Lambda$ instead, per-record signing.

[3] At DR5 with a 242 B frame under the 1% duty cycle.

[4] Under the five scope conditions of Section 3.11.

Table 17 Records per frame that each broadcast system admits with one signature per frame, by deadline, under Eq. (2). A dash means that no signature fits. Payload sizes from [22, 23].

System	Payload (B)	Rate (/s)	$D_{\max}$ 100 ms	200 ms	300 ms	1 s
ADS-B 1090ES	7	2	—	—	—	—
Remote ID, Bluetooth 4	25	1	—	—	—	—
Remote ID, Bluetooth 5	225	1	none	none	none	none
V2X CAM, 10 Hz	1500	10	none	1	2	9
V2X CAM, 1 Hz	1500	1	none	none	none	none
AUTHBC, 802.11a	1500	50	4	9	14	49

4.3 Deployment recommendations

Our findings give the following configuration guidance; the capacities behind it are simulated. (1) *High-rate 802.11 swarms* ($\Lambda D_{\max} = 5$): one Ed25519 signature and one chain link per four records, with a key record in every frame, at 43.25 B per record (per-record signing: 145.77 B); with the margin of (5), about 97.3 nodes share one collision domain. (2) *Tight deadlines*: per-record signing when $1 < \Lambda D_{\max} \leq 2$, since no batch meets the deadline. When $\Lambda D_{\max} \leq 1$ no frame meets it under the peak-age convention of Eq. (2); a deployment that tolerates one sample of staleness can use the wait $(b-1)/\Lambda$ instead. (3) *LoRaWAN provenance links*: the four feasible data rates only, with the batch sized at the record spacing it implies. On the other eight rates a signed, chained, self-verifying frame cannot be carried; Table 13 lists what each relaxation buys. (4) *Constrained receivers*: provision the processor before the channel, since the Python receiver serves 57 nodes per core, a Rust one 290, and BLS

10. (5) *Loss budget*: set ϵ at least 0.02 above the expected frame loss, since at $\epsilon = p$ the target is met with no margin (Fig. 3). (6) *Outsiders*: an 8 B group MAC, at 3.1% of capacity. Table 16 summarizes these recommendations.

4.4 Limitations

Capacities Are Simulated: On hardware, contention was measured between two radios only (Section 3.5). The capacities in Table 4 come from ns-3 with one collision domain, equal received power and static nodes. Unequal power with capture was simulated separately and raises them (Section 3.9); a swarm spread over a wider area, with hidden stations, and a moving swarm were simulated as well, with no Doppler and no time-correlated fading. Our traffic source randomizes each node's send time within its period. A real autopilot is more regular: PX4's own stream, in simulation, holds its period to about ±0.4 ms. That is nearer to the strictly periodic source, which gives the same mean capacity but a wider spread between runs (Section 2.7).

Telemetry Is Synthetic: Record sizes come from a generator. The check against flight logs covers twelve logs of one autopilot at 5 Hz position resolution, and simulated flights of that autopilot at 50 Hz, whose navigation-quality fields do not vary.

Loss Models: The loss experiment uses independent, Gilbert and length-dependent loss generated at the frame level, and collision loss traced in ns-3; no loss was measured on air. Under length-dependent loss, AUTHBC is below its target where one-record frames lose 5% (Section 3.2). Batches of two, three and eight were simulated for the capacity law alone, and nothing else here is evaluated at them.

Energy and CPU: Only the sender is metered, with a sensor never checked against a reference load; receiver and radio energy are modeled. The CPU share of the receiver counts verification and hashing, not decoding. Counting both, the Python prototype needs three of the four cores of the board for the neighborhood of AUTHBC. The Rust receiver was timed alone, without a radio or a dispatcher in the path.

Security Scope: Confidentiality, availability and key management are outside the design (Section 2.1). A swarm member holding the group key can still flood, and a delayed frame is refused only while GNSS time is available. The proofs use three separate models that are not composed, and assume canonical encoding to be injective. With one link per frame, continuity inside a frame rests on the frame signature alone, and showing one record to a third party means showing its frame. The receiver therefore keeps every accepted frame, at its on-air length for every b records held.

Post-Quantum Signatures: Falcon-512 fits a frame (Section 3.9). An ML-DSA signature is 2420 B [10] and exceeds the 1500 B MTU with any header, so it cannot travel in one frame at any batch size; freshness caps the batch at four, so it could not be amortized either. Such a signature would need a design built around fragmentation [10, 51].

Multi-Hop: Relayed records are not evaluated. A relay cannot re-sign: it carries the signatures of the originators or one BLS aggregate [15]. Both are implemented; neither is measured here.

4.5 Conclusion

What a frame must carry decides where it can be sent. On a contended broadcast channel, the cost that binds is the frame, not the authenticator. This work costs authentication at the frame rather than at the record, using measured frames, ns-3 simulation with 30 seeds per point and metered sender energy.

Two feasibility boundaries result. Where the frame fits, one signature and one chain link per four records cut the bytes per record by 70.3% and let one 802.11a collision domain carry 124 nodes where per-record signing carries 35; the batch is capped by freshness, not by the MTU, and every frame starts with a key record. Where it does not fit, eight of the twelve EU863-870 LoRaWAN data rates cannot carry one self-verifying frame at any batch size. The capacities are simulated; in a spread or moving swarm fewer neighbors are in reach (at 1 km, 8.7 with AUTHBC and 2.5 with per-record signing), but the gain stays 3.46–3.63-fold. On hardware, contention was measured between two radios only. In Rust, the frame-linked sender spends 17.1 μJ per record, a factor of 3.4 lower than the record-linked format with batching.

This accounting should carry over to other broadcast systems whose records are small beside their authenticator, although this paper evaluates it for UAV telemetry only. Future work should address the 5G NR sidelink, the 3GPP link for direct broadcast between vehicles, and a variant for post-quantum signatures that do not fit one frame.

Declarations

Author contributions. [To be completed by the authors.]

Competing interests. The authors declare no competing interests.

Appendix A AUTHBC frame layout

For reproducibility, the complete layout of the frame-linked format is given here. A frame is one canonical CBOR map with the integer keys 0–7:

```
frame = {
  0: 2,           ; v: format version
  1: 0 / 1 / 2,   ; t: frame type
  2: uint,        ; src: sender id (16 bit)
  3: uint,        ; base_seq (32 bit)
  4: 1..255,      ; n: records in frame
  5: bytes(32),   ; link: prev_hash of rec 0
  6: bytes,       ; recs: record stream
  7: bytes(64)    ; auth: one signature, or
     / [n * bytes(64)]     ; one per record
}
rec = 9 varints: ts lat lon alt
      vel_x vel_y vel_z battery mode
```

Key Implementation Notes:

- *Frame Types:* Type 0 carries one signature per record; type 1 carries one signature and decodes alone; type 2 carries one signature and codes its first record against the last record of the previous frame.

- *Record Coding:* A key record carries absolute values, with `ts`, `battery` and `mode` unsigned and the other fields zig-zag signed; a delta record carries zig-zag differences from the record before it. All values are LEB128 variable-length integers.
- *Derived Fields:* The sender identifier and sequence number of record i are `src` and `base_seq` $+\, i$. Every `prev_hash` after the first is recomputed by the receiver and never read from the frame.
- *Signed Bytes:* A signature covers the canonical CBOR encoding of the rebuilt ledger records, not the record stream.

Appendix B Why capacity is where it is: the derivation

A frame is lost in this scenario in two ways. In the first, two stations that found the medium busy draw the same backoff slot. With occupancy $\rho \approx NfT$ and a contention window of $W = 16$ slots, counting the stations that wait together gives a loss of about $\rho^2/(W(1-\rho))$. Hence $a = \rho/(W(1-\rho))$, which is 0.071 at $\rho = 0.53$, the mean occupancy at which loss reaches 5%. In the second, a station decides to send within the $4\,\mu$s in which it cannot yet sense another: that is c, the vulnerable period of [42]. Without the first-order step, the count is $\rho\,[1-(1-1/W)^{\rho/(1-\rho)}]$; with the detection term and nothing fitted, solving it for 5% gives the 18 capacities within 5.5% (mean 2.3%).

The contention model is an event simulation of the access procedure alone, with nothing fitted and none of the code of the simulator. Doubling the contention window moves the capacity of per-record signing only from 35.3 to 39.2 nodes (predicted 39.1): stations that count down for longer wait together more often.

References

[1] Farouk, M.A., Abdelaziz, M., Darwesh, A.F., Elbayoumy, A.D.: Comparative analysis of message encoding formats for blockchain-enabled flying ad-hoc networks. In: 2026 ICEENG International Conference for Innovations in Electronics, EM-Waves, and Communications (IEEC), pp. 1–6. IEEE, Cairo, Egypt (2026). https://doi.org/10.1109/IEEC69628.2026.11584026

[2] MAVLink Development Team: MAVLink Developer Guide: Message Signing (Authentication). https://mavlink.io/en/guide/message_signing.html (2026)

[3] ETSI: Intelligent transport systems (ITS); security; security header and certificate formats. Technical Specification TS 103 097, European Telecommunications Standards Institute (June 2015)

[4] Gennaro, R., Rohatgi, P.: How to sign digital streams. In: Advances in Cryptology — CRYPTO '97. LNCS, vol. 1294, pp. 180–197. Springer, Berlin, Heidelberg (1997). https://doi.org/10.1007/BFb0052235

[5] Wong, C.K., Lam, S.S.: Digital signatures for flows and multicasts. IEEE/ACM Transactions on Networking **7**(4), 502–513 (1999) https://doi.org/10.1109/90.793005

[6] Perrig, A., Canetti, R., Tygar, J.D., Song, D.: Efficient authentication and signing of multicast streams over lossy channels. In: Proc. 2000 IEEE Symposium on Security and Privacy (S&P), pp. 56–73 (2000). https://doi.org/10.1109/SECPRI.2000.848446

[7] Li, H., Shen, C., Huang, H., Wu, C.: A certificateless aggregate signature scheme for VANETs with privacy protection properties. PLOS ONE **20**(2), 0317047 (2025) https://doi.org/10.1371/journal.pone.0317047

[8] Skordoulis, D., Ni, Q., Chen, H.-H., Stephens, A.P., Liu, C., Jamalipour, A.: IEEE 802.11n MAC frame aggregation mechanisms for next-generation high-throughput WLANs. IEEE Wireless Communications **15**(1), 40–47 (2008) https://doi.org/10.1109/MWC.2008.4454703

[9] Rotta, R., Mykytyn, P.: Secure Multi-hop Telemetry Broadcasts for UAV Swarm Communication. arXiv:2401.11915 (2024)

[10] Twardokus, G., Bindel, N., Rahbari, H., McCarthy, S.: When cryptography needs a hand: Practical post-quantum authentication for V2V communications. In: Network and Distributed System Security Symposium (NDSS) (2024). https://doi.org/10.14722/ndss.2024.24267

[11] Perrig, A., Canetti, R., Tygar, J.D., Song, D.: The TESLA broadcast authentication protocol. RSA CryptoBytes **5**(2), 2–13 (2002)

[12] Veara, J., Jain, M., Moy, K., Ranganathan, A.: TBRD: TESLA Authenticated UAS Broadcast Remote ID. arXiv:2510.11343 (2025)

[13] Pannetrat, A., Molva, R.: Efficient multicast packet authentication. In: Network and Distributed System Security Symposium (NDSS) (2003)

[14] Boneh, D., Lynn, B., Shacham, H.: Short signatures from the Weil pairing. In: Advances in Cryptology — ASIACRYPT 2001. LNCS, vol. 2248, pp. 514–532. Springer, Berlin, Heidelberg (2001). https://doi.org/10.1007/3-540-45682-1_30

[15] Boneh, D., Gentry, C., Lynn, B., Shacham, H.: Aggregate and verifiably encrypted signatures from bilinear maps. In: Advances in Cryptology — EUROCRYPT 2003. LNCS, vol. 2656, pp. 416–432. Springer, Berlin, Heidelberg (2003). https://doi.org/10.1007/3-540-39200-9_26

[16] Zhang, C., Lu, R., Lin, X., Ho, P.-H., Shen, X.: An efficient identity-based batch verification scheme for vehicular sensor networks. In: IEEE INFOCOM 2008 — The 27th Conference on Computer Communications, pp. 246–250 (2008). https:

//doi.org/10.1109/INFOCOM.2008.58

[17] Rajasekaran, A.S., Maria, A., Al-Turjman, F., Altrjman, C., Mostarda, L.: Anonymous mutual and batch authentication with location privacy of UAV in FANET. Drones **6**(1), 14 (2022) https://doi.org/10.3390/drones6010014

[18] Haber, S., Stornetta, W.S.: How to time-stamp a digital document. Journal of Cryptology **3**(2), 99–111 (1991) https://doi.org/10.1007/BF00196791

[19] Mehta, P., Gupta, R., Tanwar, S.: Blockchain envisioned UAV networks: Challenges, solutions, and comparisons. Computer Communications **151**, 518–538 (2020) https://doi.org/10.1016/j.comcom.2020.01.023

[20] Gündoğan, C., Amsüss, C., Schmidt, T.C., Wählisch, M.: Reliable firmware updates for the information-centric Internet of Things. In: Proc. 8th ACM Conf. on Information-Centric Networking (ICN), pp. 59–70. ACM, New York, NY, USA (2021). https://doi.org/10.1145/3460417.3482974

[21] Darzi, S., Rahman, M.M., Karim, I., Behnia, R., Yavuz, A.A., Bertino, E.: Future-Proofing Authentication Against Insecure Bootstrapping for 5G Networks: Feasibility, Resiliency, and Accountability. arXiv:2510.23457 (2025)

[22] Card, S., Moskowitz, R.: DRIP entity tag (DET) authentication formats and protocols for broadcast remote identification (RID). RFC 9575, IETF (2024). https://doi.org/10.17487/RFC9575 . A. Wiethuechter, Ed.

[23] Strohmeier, M., Lenders, V., Martinovic, I.: On the security of the automatic dependent surveillance-broadcast protocol. IEEE Communications Surveys & Tutorials **17**(2), 1066–1087 (2015) https://doi.org/10.1109/COMST.2014.2365951

[24] Bormann, C., Hoffman, P.: Concise binary object representation (CBOR). RFC 8949, IETF (2020). https://doi.org/10.17487/RFC8949

[25] Pelkonen, T., Franklin, S., Teller, J., Cavallaro, P., Huang, Q., Meza, J., Veeraraghavan, K.: Gorilla: A fast, scalable, in-memory time series database. Proceedings of the VLDB Endowment **8**(12), 1816–1827 (2015) https://doi.org/10.14778/2824032.2824078

[26] Bianchi, G.: Performance analysis of the IEEE 802.11 distributed coordination function. IEEE Journal on Selected Areas in Communications **18**(3), 535–547 (2000) https://doi.org/10.1109/49.840210

[27] Ma, X., Chen, X.: Saturation performance of IEEE 802.11 broadcast networks. IEEE Communications Letters **11**(8), 686–688 (2007) https://doi.org/10.1109/LCOMM.2007.070040

[28] Ma, X., Chen, X.: Performance analysis of IEEE 802.11 broadcast scheme in ad hoc wireless LANs. IEEE Transactions on Vehicular Technology **57**(6), 3757–3768 (2008) https://doi.org/10.1109/TVT.2008.918731

[29] ns-3 Consortium: ns-3 Network Simulator, release ns-3.48. https://www.nsnam.org (2026)

[30] Haxhibeqiri, J., Van den Abeele, F., Moerman, I., Hoebeke, J.: LoRa scalability: A simulation model based on interference measurements. Sensors **17**(6), 1193 (2017) https://doi.org/10.3390/s17061193

[31] Zirak, Q., Shashev, D., Shidlovskiy, S.: Swarm of drones using LoRa flying ad-hoc network. In: 2021 International Conference on Information Technology (ICIT), pp. 400–405 (2021). https://doi.org/10.1109/ICIT52682.2021.9491655

[32] PX4 Development Team: PX4-Autopilot: `mavlink_main.cpp`, `configure_streams_to_default`. Source code, release v1.17.0, https://github.com/PX4/PX4-Autopilot (2026)

[33] 3GPP: Unmanned aerial system (UAS) support in 3GPP. Technical Specification TS 22.125, 3rd Generation Partnership Project (April 2022)

[34] Basin, D., Cremers, C., Dreier, J., Meier, S., Sasse, R., Schmidt, B.: The Tamarin prover. Version 1.12.0, https://tamarin-prover.com (2026)

[35] Bernstein, D.J., Duif, N., Lange, T., Schwabe, P., Yang, B.-Y.: High-speed high-security signatures. In: Cryptographic Hardware and Embedded Systems — CHES 2011. LNCS, vol. 6917, pp. 124–142. Springer, Berlin, Heidelberg (2011). https://doi.org/10.1007/978-3-642-23951-9_9

[36] National Institute of Standards and Technology: Digital signature standard (DSS). FIPS PUB 186-5, NIST (2023). https://doi.org/10.6028/NIST.FIPS.186-5

[37] Yates, R.D., Sun, Y., Brown, D.R., Kaul, S.K., Modiano, E., Ulukus, S.: Age of information: An introduction and survey. IEEE Journal on Selected Areas in Communications **39**(5), 1183–1210 (2021) https://doi.org/10.1109/JSAC.2021.3065072

[38] Josefsson, S., Liusvaara, I.: Edwards-curve digital signature algorithm (EdDSA). RFC 8032, IETF (2017). https://doi.org/10.17487/RFC8032

[39] Hesterberg, T.C.: What teachers should know about the bootstrap: Resampling in the undergraduate statistics curriculum. The American Statistician **69**(4), 371–386 (2015) https://doi.org/10.1080/00031305.2015.1089789

[40] Ma, X., Chen, X., Refai, H.H.: Performance and reliability of DSRC vehicular

safety communication: A formal analysis. EURASIP Journal on Wireless Communications and Networking **2009**, 969164 (2009) https://doi.org/10.1155/2009/969164

[41] Cao, L., Yin, H., Hu, J., Zhang, L.: Performance Analysis and Improvement on DSRC Application for V2V Communication. arXiv:2102.07023 (2021)

[42] Kleinrock, L., Tobagi, F.A.: Packet switching in radio channels: Part I—carrier sense multiple-access modes and their throughput-delay characteristics. IEEE Transactions on Communications **23**(12), 1400–1416 (1975) https://doi.org/10.1109/TCOM.1975.1092768

[43] Bianchi, G., Di Stefano, A., Giaconia, C., Scalia, L., Terrazzino, G., Tinnirello, I.: Experimental assessment of the backoff behavior of commercial IEEE 802.11b network cards. In: IEEE INFOCOM 2007 - 26th IEEE International Conference on Computer Communications, pp. 1181–1189 (2007). https://doi.org/10.1109/INFCOM.2007.141

[44] Hu, L., Dai, Z.: Performance and reliability analysis of prioritized safety messages broadcasting in DSRC with hidden terminals. IEEE Access **8**, 177112–177124 (2020) https://doi.org/10.1109/ACCESS.2020.3027550

[45] Bujari, A., Calafate, C.T., Cano, J.-C., Manzoni, P., Palazzi, C.E., Ronzani, D.: Flying ad-hoc network application scenarios and mobility models. International Journal of Distributed Sensor Networks **13**(10) (2017) https://doi.org/10.1177/1550147717738192

[46] Huang, X., Liu, A., Zhou, H., Yu, K., Wang, W., Shen, X.: FMAC: A self-adaptive MAC protocol for flocking of flying ad hoc network. IEEE Internet of Things Journal **8**(1), 610–625 (2021) https://doi.org/10.1109/JIOT.2020.3007071

[47] LoRa Alliance: RP002-1.0.3 LoRaWAN regional parameters. Technical Specification RP002-1.0.3, LoRa Alliance (2021)

[48] Semtech Corporation: SX1276/77/78/79 137–1020 MHz low power long range transceiver. Datasheet, Semtech Corporation (March 2015)

[49] Boneh, D., Gorbunov, S., Wahby, R.S., Wee, H., Wood, C.A., Zhang, Z.: BLS signatures. Internet-Draft draft-irtf-cfrg-bls-signature-05, IRTF Crypto Forum Research Group (June 2022)

[50] Wang, H., Wang, L., Zhang, K., Li, J., Luo, Y.: A conditional privacy-preserving certificateless aggregate signature scheme in the standard model for VANETs. IEEE Access **10**, 15605–15618 (2022) https://doi.org/10.1109/ACCESS.2022.3147595

[51] Sobati-Moghadam, S.: Predictive-CSM: Lightweight Fragment Security for

6LoWPAN IoT Networks. arXiv:2506.01767 (2025)